

Real-Time Incremental UAV Orthomosaicking with Translation-Only Pose Graph Optimization and View-Aware Compositing

A

Thesis Submitted

in Partial Fulfillment of the Requirements

for the Degree of

MASTER OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE

By

JUGAL ALAN

Under the Supervision of

Dr. SHASHI SHEKHAR JHA



Department of Computer Science & Engineering

Indian Institute of Technology Ropar

May 13, 2026

DECLARATION

This is to certify that the thesis entitled “**Real-Time Incremental UAV Orthomosaicking with Translation-Only Pose Graph Optimization and View-Aware Compositing**”, submitted by me to the *Indian Institute of Technology Ropar*, for the award of the degree of Master of Technology in Artificial Intelligence, is a bonafide work carried out by me under the supervision of Dr. Shashi Shekhar Jha. The content of this thesis, in full or in parts, have not been submitted to any other University or Institute for the award of any degree or diploma. I also wish to state that to the best of my knowledge and understanding nothing in this report amounts to plagiarism.

Sign: _____

Jugal Alan
Department of Computer Science & Engineering,
Indian Institute of Technology Ropar,
Rupnagar-140001, Punjab, India.

Date: _____

CERTIFICATE

This is to certify that the thesis entitled “**Real-Time Incremental UAV Orthomosaicking with Translation-Only Pose Graph Optimization and View-Aware Compositing**”, submitted by Jugal Alan (2024AIM1004), a master’s student in the *Department of Computer Science & Engineering, Indian Institute of Technology Ropar*, for the award of the degree of Master of Technology in Artificial Intelligence, is a record of an original research work carried out by the candidate under my supervision and guidance. The thesis has fulfilled all requirements as per the regulations of the institute and in my opinion has reached the standard worthy of the award of the degree. The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Sign: _____

Supervisor: Dr. Shashi Shekhar Jha
Department of Computer Science & Engineering,
Indian Institute of Technology Ropar,
Rupnagar-140001, Punjab, India.

Date: _____

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my thesis supervisor, **Dr. Shashi Shekhar Jha**, for his invaluable guidance, continuous support, and encouragement throughout this research. His insights into robotics, computer vision, and system design have been instrumental in shaping both the technical direction and the broader vision of this work.

I am grateful to the **Indian Institute of Technology Ropar** for providing the academic environment, computational resources, and institutional support that made this research possible.

I would like to thank the **Indian Army** for the opportunity to deploy and validate the system under real-world operational conditions. The field deployment provided critical feedback that significantly improved the robustness and usability of the application.

I also wish to acknowledge the open-source community - particularly the developers of OpenCV, SciPy, PyQt6, and OpenDroneMap - whose freely available tools form the foundation upon which this work is built.

Finally, I thank my family and friends for their unwavering support and patience during the course of this work.

Jugal Alan
May 2026

ABSTRACT

Real-time generation of georeferenced orthomosaics from unmanned aerial vehicle (UAV) imagery is critical for time-sensitive operations such as flood response, disaster reconnaissance, and defence surveillance. However, existing mapping pipelines either require the complete image set before processing (offline Structure-from-Motion) or depend on GPU-accelerated dense reconstruction and visual SLAM (Map2DFusion, OpenREALM), making them unsuitable for CPU-only field hardware.

This thesis presents a fully incremental orthomosaicking system that produces a continuously updated aerial map during flight, without 3D reconstruction or post-processing. The system is built on three core contributions. First, a **translation-only pose graph** refines consumer-grade GPS positions by solving sparse pairwise constraints derived from ground-projected feature correspondences. By fixing gimbal rotations from the IMU and optimizing only horizontal translations, the problem reduces to a convex linear least-squares system solved efficiently via LSQR. Second, a **view-aware compositor** combines center weighting, nadir preference, and heading-conflict arbitration to suppress double-imaging artefacts over elevated structures without requiring a Digital Surface Model. Third, a **reproducible evaluation harness** measures latency, memory consumption, seam quality, pose correction magnitude, and robustness under synthetically injected GPS drift across three diverse datasets.

Beyond the algorithmic contributions, this thesis describes two complete application systems built around the core mapping engine. A web-based mapping application (*Drone Mapper*) provides a Streamlit interface with configurable parameters, A/B comparison, and cloud deployment. A professional desktop Ground Control Station (*SkyForge GCS*), implemented as a 3100-line PyQt6 application, integrates MAVLink telemetry, live camera feeds, satellite map underlays, and mission management. The GCS has been deployed for field trials with the Indian Army.

Experimental evaluation on three datasets - covering river, village, and university campus scenes with 140 to 406 frames each - demonstrates that the system achieves 4.3 to 18.8 frames per second on a standard CPU while maintaining seam SSIM above 0.91 across all tested conditions. An ORB-based optimizer variant reduces pose-graph runtime by 4.7×

to $9.3\times$ relative to SIFT with negligible quality loss. Under synthetic GPS drift of up to 4 m, the optimizer reduces edge residuals by 4–8% without degrading mosaic quality. The system is substantially faster than offline SfM (up to $87\times$ wall-clock speedup over OpenDroneMap) while avoiding the SLAM and GPU dependencies of Map2DFusion.

Contents

List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 Motivation and Operational Context	1
1.2 Challenges in Real-Time Aerial Mapping	1
1.2.1 GPS Drift and Positional Uncertainty	2
1.2.2 The Flat-Earth Approximation	2
1.2.3 Feature-Sparse Environments	2
1.2.4 Computational Constraints	2
1.3 Research Objectives	2
1.4 Contributions	3
1.4.1 Algorithmic Contributions	3
1.4.2 Engineering Contributions	3
1.5 Thesis Organization	4
2 Literature Review	5
2.1 Offline Photogrammetric Pipelines	5
2.2 Visual SLAM for Aerial Mapping	6
2.2.1 Feature-Based Visual SLAM	6
2.2.2 SLAM-Based Aerial Mapping	6
2.3 Direct Georeferencing	6
2.4 Pose-Graph Optimization in Robotics and Aerial Mapping	7
2.5 Image Compositing and Seam Handling	7
2.5.1 Multi-Band Blending	8
2.5.2 Seam-Finding Algorithms	8
2.5.3 Score-Based Arbitration	8
2.6 Comparative Summary	8
3 Core Mapping Engine	11
3.1 System Architecture Overview	11
3.2 Reference Frames and Camera Model	11
3.3 Telemetry-Based Pose Extraction	13
3.3.1 Metadata Extraction	13
3.3.2 GPS-to-Local Conversion	13
3.3.3 Rotation from Gimbal Angles	13
3.4 Inverse Pinhole Projection	14

3.4.1	Ground Plane Considerations	14
3.5	Dynamic Sparse Tiling	15
3.5.1	Key Properties	15
3.6	Multi-Band Laplacian Pyramid Blending	15
3.6.1	Pyramid Construction	15
3.6.2	Weight Mask	16
3.6.3	Max-Weight Tile Update	17
3.6.4	Map Reconstruction	18
3.7	Coverage Tracking and Quality Metrics	19
3.7.1	Intersection-over-Union	19
3.7.2	Flight Path Tracking	19
3.7.3	Coverage Area Estimation	19
3.7.4	Seam SSIM	19
4	Pose Graph Optimization and View-Aware Compositing	21
4.1	Translation-Only Pose Graph	21
4.1.1	Problem Formulation	21
4.1.2	Candidate Pair Selection	22
4.1.3	View Compatibility Filtering	23
4.1.4	Ground-Plane Pairwise Constraints	23
4.1.5	Parallax Screening by Spatial Rigidity	24
4.1.6	Sparse Translation-Only Optimization	24
4.1.7	Solve Strategy	25
4.1.8	Feature Detector Choice: SIFT vs. ORB	25
4.2	View-Aware Incremental Compositing	26
4.2.1	View Metrics	26
4.2.2	Heading-Conflict Detection	26
4.2.3	Overlap Arbitration Rules	26
4.2.4	Rationale and Limitations	27
5	Applications and Deployment	29
5.1	Drone Mapper: Web-Based Mapping Application	29
5.1.1	Interface Design	29
5.1.2	A/B Comparison Framework	30
5.1.3	Cloud Deployment	30
5.2	SkyForge Ground Control Station	30
5.2.1	Architecture	31
5.2.2	User Interface	32
5.2.3	Video Frame Intelligence	32
5.2.4	Satellite Map Integration	33
5.2.5	Configuration and Persistence	33
5.3	MAVLink Deployment Pipeline	33
5.3.1	Hardware Chain	33
5.3.2	Telemetry Acquisition	33
5.3.3	DJI-Compatible XMP Injection	35
5.3.4	Camera Capture	35
5.3.5	Operational Modes	35

5.3.6	Flight Data Logging	35
5.4	Field Deployment	35
6	Experimental Evaluation and Conclusion	39
6.1	Datasets and Experimental Setup	39
6.2	Runtime Performance	39
6.3	View-Aware Blending Evaluation	40
6.4	Mosaic Quality Analysis	40
6.5	Pose-Graph Analysis	42
6.6	Robustness Under Synthetic GPS Drift	43
6.6.1	Fixed-Bias Stress Test	43
6.6.2	Multi-Drift Robustness	43
6.7	Hyperparameter Sensitivity	43
6.7.1	GPS Prior Weight	44
6.7.2	Match Weight	44
6.7.3	Feature Count	44
6.7.4	Solve Interval	45
6.8	Comparison with Existing Pipelines	45
6.9	Discussion	45
6.10	Conclusion	46
6.11	Future Work	47
	 Bibliography	 49

List of Figures

3.1	High-level architecture of the proposed real-time mapping system, showing the flow of data from UAV telemetry to the Multi-Band Blending Engine.	12
3.2	Two drones independently map distinct spatial segments, fusing their warped frames into a unified global mosaic.	16
3.3	Resulting orthomosaic from the phantom3-village dataset showing successful stitching.	17
3.4	Visual representation of the Gaussian and Laplacian decomposition levels.	18
4.1	Pose graph optimization pipeline: IoU-filtered SIFT matching builds visual constraints, which are jointly solved with GPS priors via sparse LSQR.	22
4.2	Relief-displacement artifacts before and after the proposed translation refinement and view-aware arbitration.	27
5.1	Extended system architecture: Pose Graph Optimizer, Streamlit UI, cloud deployment, and MAVLink live feed pipeline.	31
5.2	MAVLink deployment pipeline data flow: Pixhawk → Skydroid Air Unit → MK15 → ADB → XMP injection → mapping engine.	34
5.3	Preparation and field deployment of the SkyForge GCS with the Indian Army.	36
5.4	Real-time mapping in progress during field trials.	36
5.5	SkyForge UI during operation, displaying the generated orthomosaic over a satellite underlay.	37
5.6	SkyForge UI during operation without the satellite underlay, isolating the drone-generated map.	37
6.1	A/B comparison on IIT Ropar flights: (left) raw GPS with visible tile-boundary seams, (right) pose-graph-optimized with improved alignment.	41
6.2	Orthomosaic from DJI Mavic 3T flights over Ropar area, with optimized flight path overlay.	41

List of Tables

2.1	Comparison of aerial mapping architectures.	9
3.1	Three-layer system architecture.	11
6.1	Dataset characteristics.	40
6.2	Per-frame runtime on clean trajectories (AMD Ryzen 7 7840HS, 0.5 m/px).	40
6.3	Effect of view-aware overlap arbitration on blending quality.	41
6.4	Mosaic quality metrics on clean trajectories.	42
6.5	Pose-graph statistics on clean trajectories.	42
6.6	Synthetic GPS bias experiment on data1 (3 m linear bias).	43
6.7	Robustness under varying synthetic drift profiles (data1, ORB optimizer).	43
6.8	Effect of GPS prior weight w_{gps} (data1, ORB, 3 m bias, $w_{\text{match}} = 0.25$).	44
6.9	Effect of match weight w_{match} (data1, ORB, 3 m bias, $w_{\text{gps}} = 9.0$).	44
6.10	Effect of feature count on optimizer runtime and quality (data1, clean trajectory).	45
6.11	Effect of solve interval on runtime and correction (data1, clean, ORB).	45
6.12	Comparison with existing pipelines. Our system reports the Opt. ORB variant.	46

Chapter 1

Introduction

1.1 Motivation and Operational Context

Operators of unmanned aerial vehicles (UAVs) frequently need a georeferenced map while the aircraft is still airborne. Applications range from flood disaster response - where the “golden hour” of emergency operations depends on immediate spatial intelligence - to defence reconnaissance, agricultural survey, and infrastructure monitoring. In all these scenarios, the ability to generate a continuously updated orthomosaic during flight transforms the UAV from a data-collection platform into a real-time situational awareness tool.

The dominant mapping pipeline - Structure-from-Motion (SfM) followed by global bundle adjustment and dense orthorectification - produces high-quality products but requires the complete image set before processing can begin [1–3]. Processing times range from minutes to hours depending on the dataset size, turning the output into a historical document rather than a tactical asset. This latency is unacceptable when floodwaters are actively reshaping the landscape, when commanders need immediate terrain awareness, or when time-critical decisions depend on up-to-the-second mapping.

Real-time alternatives must commit to a result frame by frame, tolerate imperfect GPS, and - critically for field deployment - run on CPU-only hardware. GPU-accelerated pipelines such as Map2DFusion [4] and OpenREALM [5] produce high-quality local maps but depend on visual SLAM and dense reconstruction, which can fail under large baselines, sparse texture, or the featureless conditions typical of flood-affected areas. GPS-only mosaicking avoids SLAM entirely but leaves positional drift uncorrected, producing visible misalignments at seam boundaries.

This thesis addresses both the geometric and compositing sides of real-time orthomosaicking, and additionally develops the engineering infrastructure required to deploy such a system in real-world operations.

1.2 Challenges in Real-Time Aerial Mapping

Several interrelated challenges must be addressed to build a practical real-time mapping system:

1.2.1 GPS Drift and Positional Uncertainty

Consumer-grade GNSS receivers on commercial drones exhibit position errors of 2–5 metres, with larger errors during changes in heading or altitude. A 1° boresight misalignment at 100 m altitude causes approximately 1.7 m of lateral displacement in the projected image. Without correction, these errors accumulate as visible seam discontinuities in the incremental mosaic.

1.2.2 The Flat-Earth Approximation

Projecting aerial images onto a horizontal ground plane ($Z = 0$) is computationally efficient but produces relief displacement artefacts near elevated structures. Buildings and trees that are far from the nadir point appear to “lean” outward. When opposing flight directions view the same structure, this produces double-imaging - a particularly disruptive artefact that simple score-based compositing cannot resolve.

1.2.3 Feature-Sparse Environments

Standard feature-matching algorithms (SIFT, SURF, ORB) assume a textured, rigid scene. Water surfaces violate these assumptions through feature starvation (turbid floodwaters lack stable textures), specular reflection (detected features are moving reflections), and dynamic deformation (texture patches change or disappear between frames). Any real-time system intended for flood response must function gracefully when visual features are unavailable.

1.2.4 Computational Constraints

Field deployment hardware ranges from laptops to embedded systems, rarely with dedicated GPUs. A practical system must achieve real-time throughput on CPU-only platforms while processing high-resolution imagery (4000×3000 pixels from DJI drones). This constraint eliminates approaches based on dense reconstruction or deep learning inference without careful engineering.

1.3 Research Objectives

This thesis pursues the following research objectives:

1. **Design a translation-only pose graph** that corrects GPS drift by solving pairwise ground-projected feature constraints against a GPS prior via sparse least-squares. By fixing gimbal rotations and optimizing only horizontal translations, the problem remains strictly linear with bounded per-frame cost.
2. **Develop a view-aware compositor** that detects heading conflicts - pixels where incoming and stored imagery originate from opposing flight directions - and arbitrates them using a per-pixel nadir score, suppressing lean and double-façade artefacts without requiring a Digital Surface Model.

3. **Build a reproducible evaluation harness** that measures latency, memory, seam quality, pose correction, and robustness under synthetic GPS drift on multiple diverse datasets.
4. **Engineer complete application systems** - a web-based mapping tool and a professional desktop Ground Control Station - that demonstrate the core engine's readiness for real-world deployment, including live MAVLink telemetry integration.

1.4 Contributions

The contributions of this thesis are organized into algorithmic and engineering components:

1.4.1 Algorithmic Contributions

- A **parallax-aware translation pose graph** that filters candidate pairs by view compatibility, enforces spatial rigidity constraints, and solves a convex least-squares problem via LSQR. The formulation is shown to be stable under synthetic GPS drift of up to 4 m while adding only 27–80 ms per frame with ORB features.
- A **view-aware overlap arbitration scheme** that combines frame-level nadir scoring with per-pixel heading-conflict detection. On the PhantomVillage dataset, the system detects over 2.8 million conflict pixels and overrides 278,658 of them based on nadir preference, suppressing double-imaging artefacts.
- A **comprehensive experimental evaluation** comparing three processing variants (Raw GPS, Optimized SIFT, Optimized ORB) across three datasets, including comparison with OpenDroneMap and Map2DFusion, and sensitivity analysis of six hyperparameters.

1.4.2 Engineering Contributions

- **Drone Mapper**: A web-based application built on Streamlit with configurable parameters, A/B comparison mode, interactive map visualization, and cloud deployment on DigitalOcean.
- **SkyForge GCS**: A professional desktop Ground Control Station implemented as a 3100-line PyQt6 application with MAVLink telemetry integration, live camera feed processing, satellite map underlays, dark/light theming, and mission management. The GCS has been deployed for field trials with the Indian Army.
- **MAVLink deployment pipeline**: A hardware-to-software bridge connecting Pixhawk flight controllers through Skydroid telemetry links to the mapping engine, with DJI-compatible XMP metadata injection for cross-platform airframe compatibility.

1.5 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 reviews the related work across offline photogrammetry, visual SLAM, direct georeferencing, pose-graph optimization, and image compositing, establishing the context and gaps that this work addresses.

Chapter 3 describes the core mapping engine: coordinate systems, telemetry-based pose extraction, inverse pinhole projection, dynamic sparse tiling, and multi-band Laplacian pyramid blending.

Chapter 4 presents the two main algorithmic contributions: the translation-only pose graph with parallax screening, and the view-aware incremental compositor with heading-conflict arbitration.

Chapter 5 describes the two application systems built around the core engine - the Drone Mapper web application and the SkyForge Ground Control Station - including the MAVLink deployment pipeline and field deployment.

Chapter 6 presents the experimental evaluation across three datasets, including runtime performance, mosaic quality analysis, pose-graph statistics, robustness under synthetic drift, hyperparameter sensitivity studies, comparison with existing pipelines, and concludes with a summary of findings and directions for future work.

Chapter 2

Literature Review

This chapter reviews the principal research areas that inform the design of a real-time UAV orthomosaicking system. The discussion is organized from the most established (and computationally expensive) approaches to the most lightweight, culminating in a comparative summary that identifies the gap addressed by this thesis.

2.1 Offline Photogrammetric Pipelines

Offline photogrammetry remains the gold standard for producing centimetre-accurate orthomosaics and Digital Elevation Models (DEMs). The standard pipeline comprises feature extraction, pairwise matching, incremental or global Structure-from-Motion (SfM), dense multi-view stereo (MVS) reconstruction, mesh generation, and orthorectification.

COLMAP [1] implements a complete SfM and MVS pipeline with robust feature matching, geometric verification, and global bundle adjustment. Snavely et al. [6] demonstrated the scalability of SfM to large unstructured photo collections.

OpenDroneMap (ODM) [2] provides an open-source end-to-end pipeline specifically designed for drone imagery, including GPS-based initial pose estimation, feature extraction, point cloud densification, and orthomosaic generation. Processing times for typical drone surveys (200–400 images) range from 5 minutes to several hours depending on configuration.

Pix4D [3] and **Agisoft Metashape** [7] are commercial photogrammetry solutions offering optimized implementations with GPU acceleration, automatic ground control point detection, and quality reports.

The computational bottleneck of these pipelines is **bundle adjustment (BA)**, which jointly optimizes all camera poses and 3D structure by minimizing total reprojection error. The normal equations of BA lead to a system whose computational cost scales as $O(n^3)$ in the number of cameras for dense methods, or $O(n \cdot m^2)$ for sparse methods where m is the average number of observations per camera. For real-time applications, this batch processing requirement is the fundamental limitation: reconstruction cannot begin until the complete dataset is available, and the processing time far exceeds mission duration.

2.2 Visual SLAM for Aerial Mapping

Simultaneous Localization and Mapping (SLAM) addresses the latency of SfM by treating mapping as an incremental problem, processing each frame as it arrives and maintaining a map that grows over time.

2.2.1 Feature-Based Visual SLAM

ORB-SLAM3 [8] supports monocular, stereo, and visual-inertial configurations with relocalization and multi-map capabilities. It maintains a sparse map of ORB features and performs local bundle adjustment for each keyframe. However, the resulting maps are sparse point clouds designed for robot localization, not the dense, human-readable orthomosaics needed by operators.

VINS-Mono [9] tightly couples visual features with IMU pre-integration for robust state estimation. While it provides excellent trajectory accuracy, it does not produce orthoimages.

2.2.2 SLAM-Based Aerial Mapping

Several systems have combined SLAM with image compositing specifically for UAV mapping:

Map2DFusion [4] couples monocular SLAM with an adaptive multi-band compositor to produce orthoimages in real time. The system uses GPU-accelerated dense reconstruction and requires a visual odometry front-end. While it demonstrates impressive results on village-scale scenes, the dependency on GPU hardware and reliable visual tracking limits field deployment on constrained platforms.

OpenREALM [5] modularizes the SLAM-based mapping approach with interchangeable SLAM back-ends (ORB-SLAM2, SVO) and produces georeferenced orthomosaics with DSM support. Like Map2DFusion, it depends on GPU hardware for real-time operation.

The common limitation of SLAM-based mapping systems for our target application is threefold: (1) visual tracking can fail under large baselines, sparse texture, or over water surfaces; (2) GPU hardware is required for real-time dense reconstruction; and (3) the SLAM front-end introduces additional complexity and failure modes.

2.3 Direct Georeferencing

Direct georeferencing bypasses feature-based pose estimation entirely, using the camera position and orientation directly from onboard GNSS and IMU sensors [10, 11]. The image is projected onto a reference surface using a homography derived from sensor metadata.

The principal advantage is computational: each frame's placement depends only on its metadata and not on its relationship to other images, reducing per-frame complexity

to $O(1)$. This makes direct georeferencing inherently suitable for real-time incremental operation. The system presented in this thesis uses direct georeferencing as the primary pose source.

The principal limitation is that map accuracy is strictly coupled to sensor fidelity. Consumer-grade GNSS modules exhibit position errors of 2–5 m, and uncalibrated bore-sight angles introduce systematic offset. Unlike feature-based methods, there is no feedback mechanism to correct these errors. This limitation motivates the pose-graph correction layer described in Chapter 4.

2.4 Pose-Graph Optimization in Robotics and Aerial Mapping

Pose-graph optimization is well established in robotics SLAM, where nodes represent robot poses and edges encode relative-pose measurements [12, 13]. The optimization finds the set of poses that best satisfies all constraints in a least-squares sense.

Kümmerle et al. [12] introduced the **g2o** framework for general graph optimization, demonstrating that pose graphs can be solved efficiently using sparse matrix factorizations. The standard formulation optimizes a full 6-DoF pose (3D rotation + 3D translation), which requires a good initial guess and can diverge when the constraint graph is sparse.

In aerial mapping, similar graphs have been used to distribute GPS drift across keyframes. However, the typical UAV mapping scenario differs from ground-robot SLAM in several important ways:

- **Reliable orientation:** UAV gimbals provide accurate attitude estimates, making rotational optimization unnecessary for map-level accuracy.
- **Planar motion:** At survey altitudes, the dominant positional error is horizontal GPS drift; vertical errors are partially absorbed by the flat-earth projection.
- **Sparse connectivity:** Unlike ground robots that revisit areas frequently, survey flights produce sparse graphs with limited loop closure opportunities.

These observations motivate the *translation-only* formulation adopted in this thesis: by fixing gimbal rotations and optimizing only horizontal corrections, the problem reduces to a convex linear least-squares system that admits an exact solution without iterative nonlinear optimization.

2.5 Image Compositing and Seam Handling

The final stage of orthomosaic production is compositing overlapping warped images into a seamless output. Several approaches exist:

2.5.1 Multi-Band Blending

Burt and Adelson [14] introduced Laplacian pyramid blending, which decomposes images into frequency bands and blends each band independently. Low-frequency components (exposure variations) are blended over wide transition zones, while high-frequency components (edges, textures) are blended sharply. Brown and Lowe [15] applied this to automatic panoramic stitching with SIFT features.

Multi-band blending is the standard approach for hiding seams in panoramic stitching and is used by most photogrammetry pipelines. However, conventional multi-band blending assumes a single compositing pass over the full image set and is not directly applicable to incremental updates.

2.5.2 Seam-Finding Algorithms

Graph-cut and dynamic programming approaches search for optimal cut paths through overlap regions to minimize visible discontinuities. These methods produce high-quality results but require global optimization over the overlap area, making them computationally expensive for incremental operation.

2.5.3 Score-Based Arbitration

An alternative to blending is per-pixel arbitration, where the “best” observation for each pixel is selected based on a quality score. Common scoring criteria include distance from image center (center-weighted), viewing angle (nadir preference), and temporal recency.

The compositor presented in this thesis extends score-based arbitration with *heading-conflict detection*, a mechanism for identifying pixels where incoming and stored imagery originate from substantially different flight directions. This addresses the specific artefact pattern caused by opposing-direction views over elevated structures - a problem not handled by standard score-based methods.

2.6 Comparative Summary

Table 2.1 summarizes the key differences between the major mapping approaches and the system proposed in this thesis.

The proposed system occupies a distinct position in this design space: it provides real-time incremental orthomosaics on CPU-only hardware, uses visual features for pose correction rather than for tracking, and degrades gracefully to pure direct georeferencing when features are unavailable. This combination of properties makes it uniquely suited for field deployment in resource-constrained environments.

TABLE 2.1: Comparison of aerial mapping architectures.

Approach	Examples	Strengths	Weaknesses	Real-Time?
SfM Photogrammetry	COLMAP, ODM, Metashape	Very high accuracy; produces 3D models and DEMs; global optimization.	Slow (minutes to hours); feature-dependent; batch processing only.	No
Visual SLAM	ORB-SLAM3, VINS-Mono	Low latency; incremental tracking; loop closure.	Sparse maps; tracking drift; fragile over water; GPU for dense output.	Limited
SLAM + Compositor	Map2DFusion, OpenREALM	Real-time orthomosaics with DSM support.	Requires GPU; visual SLAM front-end can fail.	Yes (GPU)
Direct Georef. + Pose Graph	This Thesis	CPU-only real-time (≥ 4 FPS); robust over water; lightweight pose correction; multi-UAV ready.	Accuracy limited by GPS/IMU quality; flat-earth assumption; no 3D output.	Yes (CPU)

Chapter 3

Core Mapping Engine

This chapter describes the foundational components of the mapping system: the coordinate frames, camera model, telemetry-based pose extraction, inverse pinhole projection, dynamic sparse tiling, and multi-band Laplacian pyramid blending. Together, these components enable deterministic, incremental orthomosaic generation at $O(1)$ computational cost per frame.

3.1 System Architecture Overview

The system is organized into three decoupled layers, enabling independent development and deployment:

TABLE 3.1: Three-layer system architecture.

Layer	Components
Engine	PoseExtractor, PoseGraphOptimizer, MultiBandMap2D (tiled blending engine)
Service	MapperService singleton, processing pipeline, metrics collection, performance profiling
Frontend	Application interface (Streamlit web UI or PyQt6 desktop GCS)

The engine layer contains no application-specific logic and can be driven by any front-end. The service layer manages the processing pipeline, coordinates the pose extractor and optimizer with the mapping engine, and collects performance metrics. The front-end layer handles user interaction, visualization, and deployment-specific concerns. This separation allows the same core engine to power both the web-based Drone Mapper application and the desktop SkyForge GCS described in Chapter 5.

3.2 Reference Frames and Camera Model

Three coordinate frames are used throughout the pipeline:

Geodetic frame. Global reference using WGS-84 latitude, longitude, and ellipsoidal altitude. Raw GNSS measurements from the drone are expressed in this frame.

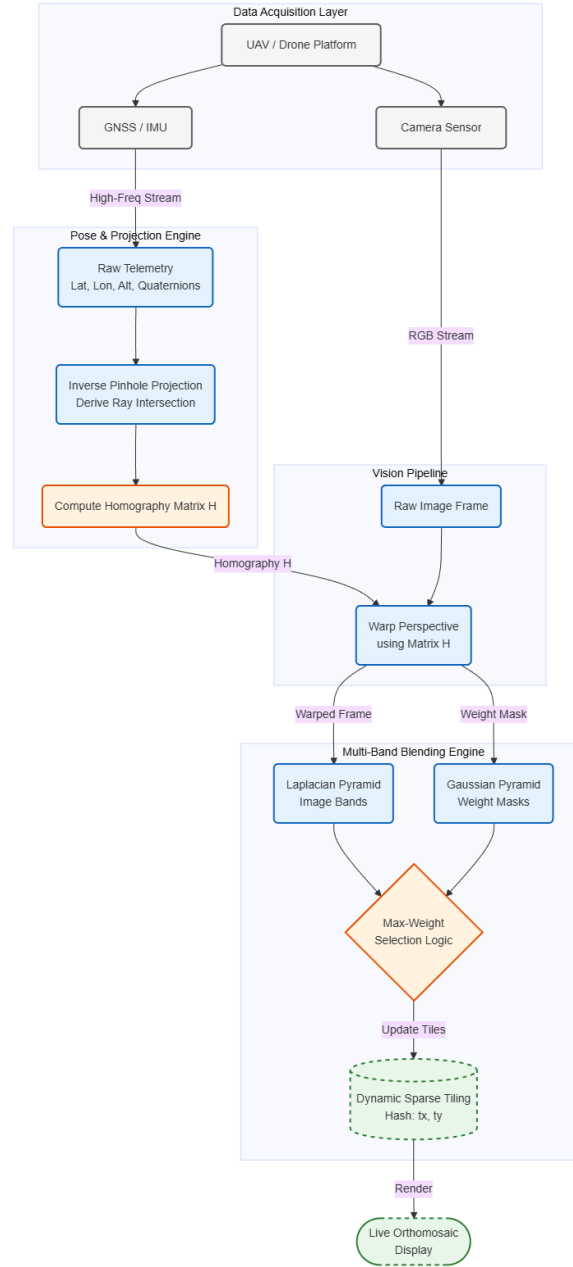


FIGURE 3.1: High-level architecture of the proposed real-time mapping system, showing the flow of data from UAV telemetry to the Multi-Band Blending Engine.

Local NED frame. Geographic positions are first converted from latitude and longitude to Universal Transverse Mercator (UTM) coordinates using the PyProj library. A local North-East-Down (NED) frame is then defined using the first valid image position as origin. In this frame, the X-axis points north, the Y-axis points east, and the Z-axis points downward. Consequently, a camera at positive relative altitude h_i is represented with negative NED height, i.e., $t_{i,z} = -h_i$.

Camera frame. The camera frame follows the standard computer-vision convention with X_c to the right, Y_c downward, and Z_c forward (optical axis). The camera intrinsics

are represented by

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \quad (3.1)$$

In the current implementation, f_x and f_y are approximated from the focal length, sensor width, and image width, and the principal point is assumed to lie at the image centre. Lens distortion is not modelled explicitly.

3.3 Telemetry-Based Pose Extraction

The `PoseExtractor` class parses DJI XMP metadata directly from JPEG bytes to extract camera pose. This approach avoids external dependencies such as `ExifTool` and operates on raw image bytes, making it suitable for both file-based and streaming workflows.

3.3.1 Metadata Extraction

Each input image is parsed to extract DJI XMP metadata fields: `GpsLatitude`, `GpsLongitude`, `RelativeAltitude`, `GimbalRollDegree`, `GimbalPitchDegree`, and `GimbalYawDegree`. The extractor searches for the XMP packet within the JPEG bytes using string matching for the `drone-dji` namespace, then applies regular expressions to extract individual field values.

3.3.2 GPS-to-Local Conversion

Let (E_i, N_i) denote the UTM coordinates of image i , and let (E_0, N_0) be the origin defined by the first valid image. The UTM zone is determined automatically from the first image's longitude. The initial translation in the local NED frame is:

$$\mathbf{t}_i^{\text{gps}} = \begin{bmatrix} N_i - N_0 \\ E_i - E_0 \\ -h_i \end{bmatrix}. \quad (3.2)$$

3.3.3 Rotation from Gimbal Angles

The reported gimbal yaw, pitch, and roll angles are converted into a camera-to-world rotation in the NED frame. A fixed base transformation aligns the zero-angle camera orientation (camera looking north, horizon-level) with the NED convention:

$$\mathbf{R}_{\text{base}} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}. \quad (3.3)$$

This matrix maps the camera's forward direction ($+Z_c$) to north ($+X_{\text{NED}}$), the camera's right direction ($+X_c$) to east ($+Y_{\text{NED}}$), and the camera's down direction ($+Y_c$) to down ($+Z_{\text{NED}}$).

The DJI gimbal convention defines: yaw ψ as rotation about the down axis (positive clockwise from north), pitch θ as rotation about the east axis (0° = horizon, -90° = nadir), and roll γ as rotation about the north axis. The initial rotation is then:

$$\mathbf{R}_i^0 = \mathbf{R}_z(\psi_i) \mathbf{R}_y(\theta_i) \mathbf{R}_x(\gamma_i) \mathbf{R}_{\text{base}}, \quad (3.4)$$

where the rotation matrices follow the intrinsic (body-frame) convention.

The resulting initial camera pose is:

$$\mathbf{T}_i^0 = \begin{bmatrix} \mathbf{R}_i^0 & \mathbf{t}_i^{\text{gps}} \\ \mathbf{0}^\top & 1 \end{bmatrix}. \quad (3.5)$$

The rotational component is treated as fixed throughout the pipeline. Only the horizontal translation is subsequently refined by the pose-graph optimizer (Chapter 4).

3.4 Inverse Pinhole Projection

Each incoming frame is projected onto the ground plane using the inverse of the pinhole camera model. A pixel $\mathbf{p} = (u, v, 1)^\top$ is back-projected to a normalized camera ray:

$$\mathbf{r}_c = \mathbf{K}^{-1} \mathbf{p}. \quad (3.6)$$

The ray is normalized and transformed to the world frame:

$$\mathbf{r}_w = \mathbf{R}_i \frac{\mathbf{r}_c}{\|\mathbf{r}_c\|_2}. \quad (3.7)$$

The ground-plane intersection is obtained by ray casting to $Z = 0$ (or a specified plane height):

$$\lambda = -\frac{t_{i,z}}{r_{w,z}}, \quad \mathbf{P}_i = \mathbf{t}_i + \lambda \mathbf{r}_w. \quad (3.8)$$

Projecting the four image corners produces a quadrilateral footprint in metric map coordinates. A homography $\mathbf{H}$ is then computed between the image corners and their ground-plane projections:

$$\mathbf{x}' = \mathbf{H} \mathbf{x}. \quad (3.9)$$

This homography enables fast, deterministic warping via OpenCV's `warpPerspective`. The entire projection pipeline - from pixel coordinates to ground-plane positions - requires only matrix multiplications and a single division, with no iterative optimization.

3.4.1 Ground Plane Considerations

The flat-earth assumption ($Z = 0$) is valid for typical UAV survey altitudes (50–200 m) over terrain with moderate relief variation. For flood scenarios, the water surface is approximately planar, and the inverse projection relies only on telemetry geometry rather than image content, providing stability over reflective or textureless surfaces.

For scenes with significant elevation variation, the relief displacement error is:

$$\Delta d = \frac{\Delta h \cdot r}{H}, \quad (3.10)$$

where Δh is the height difference from the assumed ground plane, r is the radial distance from the nadir point, and H is the flight altitude. At 120 m altitude, a 15 m building at the image edge produces approximately 3–5 m of displacement.

3.5 Dynamic Sparse Tiling

The global mosaic is stored as a sparse collection of fixed-size tiles rather than a single monolithic image. Each tile is identified by integer coordinates:

$$(t_x, t_y) = \left(\left\lfloor \frac{X}{L} \right\rfloor, \left\lfloor \frac{Y}{L} \right\rfloor \right), \quad (3.11)$$

where $L = \text{tile_size} \times \text{resolution}$ is the tile extent in metres (default: $512 \times 0.5 = 256$ m). Tiles are stored in a Python dictionary, providing $O(1)$ access by coordinate.

3.5.1 Key Properties

- **Lazy allocation:** Only tiles touched by warped images are ever allocated. Memory grows proportionally to the explored area, not to the number of processed frames or elapsed time.
- **Multi-UAV compatibility:** Multiple UAV streams naturally contribute to different spatial tiles without coordination, enabling thread-safe concurrent updates via a single global lock.
- **Scalability:** For a 500×500 m survey area at 0.5 m/px resolution, the tile grid requires approximately 150 MB including Laplacian pyramid storage. Larger missions can be supported by implementing LRU eviction of cold tiles to disk.

3.6 Multi-Band Laplacian Pyramid Blending

When geometric alignment is imperfect, naive averaging of overlapping images causes “ghosting.” The system uses Laplacian pyramid blending [14] to handle exposure differences at low frequencies while preserving sharp edges at high frequencies.

3.6.1 Pyramid Construction

The Gaussian pyramid G represents the image at progressively coarser scales:

$$G_0 = I, \quad G_{i+1} = \text{Downsample}(G_i). \quad (3.12)$$

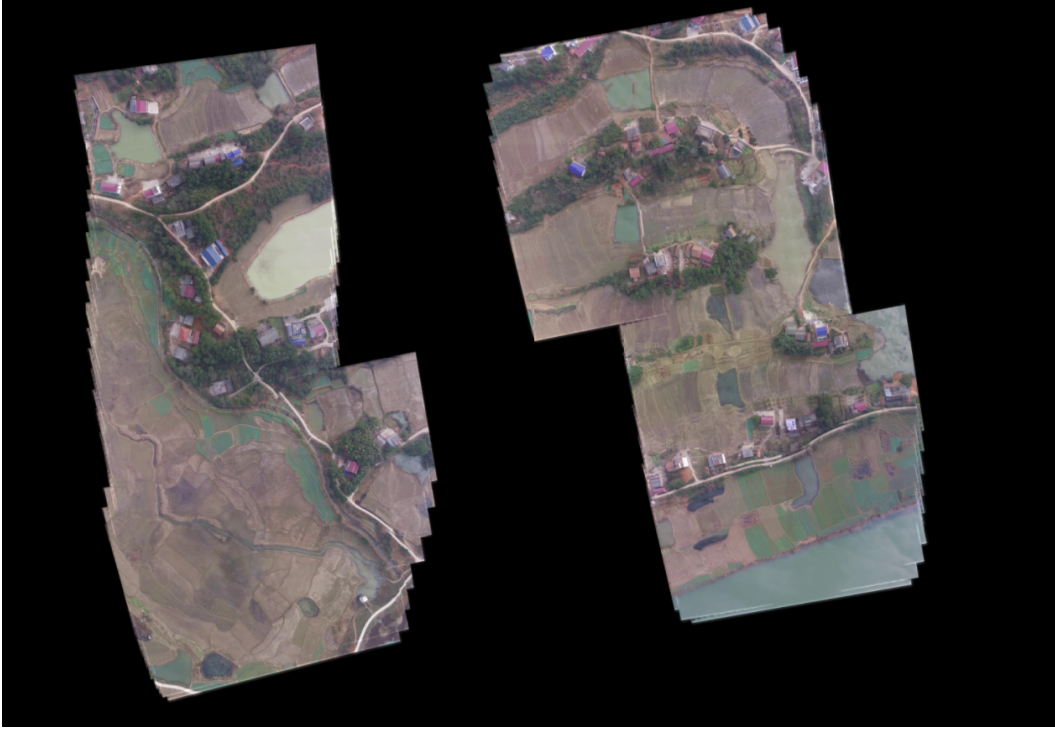


FIGURE 3.2: Two drones independently map distinct spatial segments, fusing their warped frames into a unified global mosaic.

The Laplacian pyramid L captures the detail lost at each downsampling step:

$$L_i = G_i - \text{Upsample}(G_{i+1}). \quad (3.13)$$

The system uses B pyramid levels (default $B = 2$), plus the residual low-frequency image G_B .

3.6.2 Weight Mask

A radial weighting function prioritizes pixels near the image centre, where geometric distortion is minimal:

$$w(u, v) = \left(1 - \frac{\sqrt{(u - c_x)^2 + (v - c_y)^2}}{R_{\max}} \right)^2, \quad (3.14)$$

where (c_x, c_y) is the image centre, $R_{\max}$ is the distance to the corner, and the squaring provides stronger centre bias. This mask is computed once and reused across frames of the same resolution.

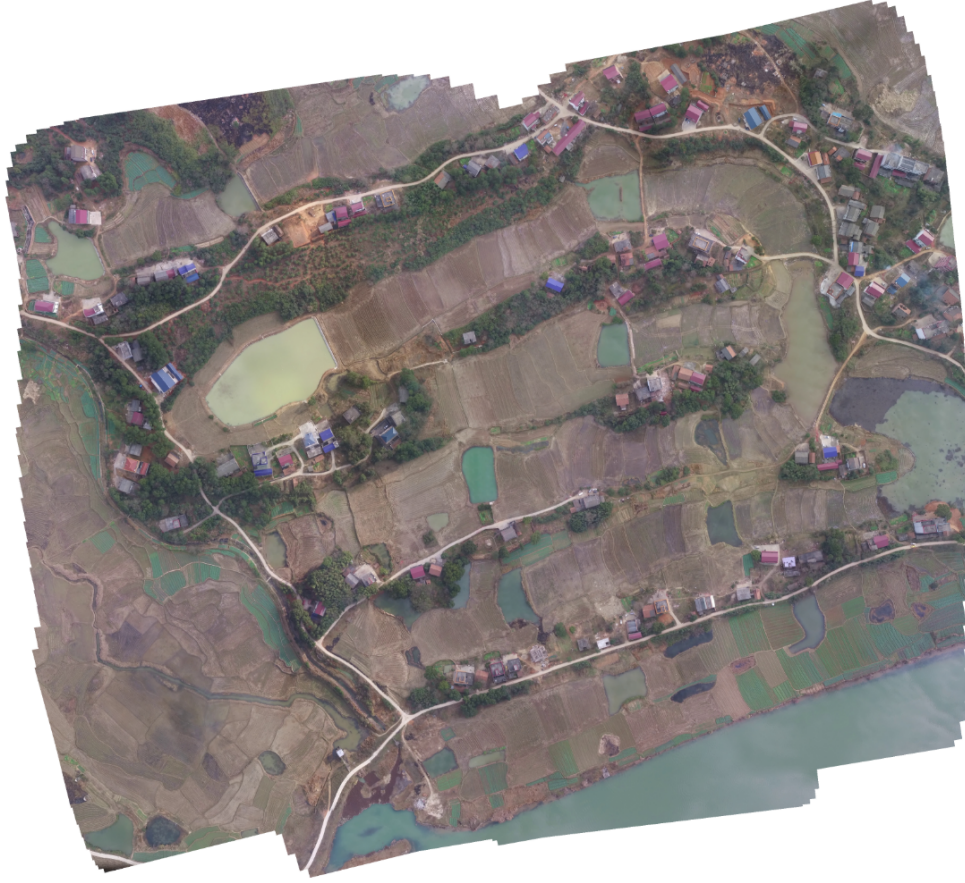


FIGURE 3.3: Resulting orthomosaic from the phantom3-village dataset showing successful stitching.

3.6.3 Max-Weight Tile Update

For each tile touched by the incoming frame, the system applies a max-weight selection rule at each pyramid level:

$$L_{\text{map}}(x, y) = \begin{cases} L_{\text{new}}(x, y) & \text{if } W_{\text{new}}(x, y) > W_{\text{map}}(x, y) \\ L_{\text{map}}(x, y) & \text{otherwise} \end{cases} \quad (3.15)$$

This “closest-to-nadir” approach eliminates ghosting by enforcing a pixel-level decision for high-frequency details. The Laplacian structure ensures that transitions remain smooth at low frequencies.

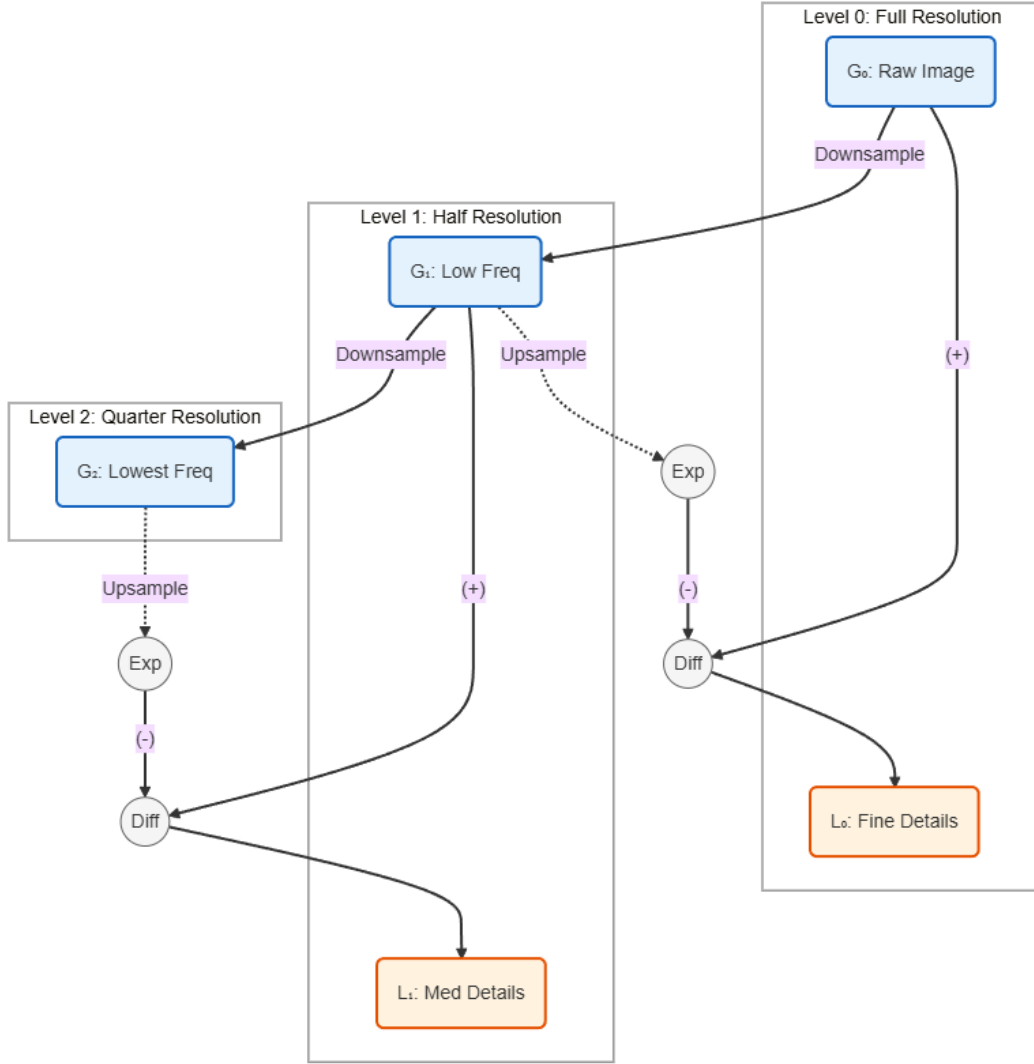


FIGURE 3.4: Visual representation of the Gaussian and Laplacian decomposition levels.

3.6.4 Map Reconstruction

The engine stores only Laplacian pyramid layers for each tile. Full reconstruction occurs on demand:

$$I = \sum_{i=0}^{B-1} (\text{Upsample}(L_{i+1}) + L_i) . \quad (3.16)$$

This lazy reconstruction strategy - analogous to contemporary GIS tile servers - minimizes computational cost during the time-critical feed operation and defers the more expensive reconstruction to visualization time.

3.7 Coverage Tracking and Quality Metrics

3.7.1 Intersection-over-Union

The system computes the Intersection-over-Union (IoU) between consecutive frame footprints as a measure of overlap:

$$\text{IoU}(i,j) = \frac{|\text{footprint}_i \cap \text{footprint}_j|}{|\text{footprint}_i \cup \text{footprint}_j|}. \quad (3.17)$$

The IoU is computed via rasterized polygon masks at 0.1 m resolution. This metric serves both as input to the pose-graph optimizer (for candidate pair selection) and as a quality indicator for the operator.

3.7.2 Flight Path Tracking

Camera positions are stored as (east, north) tuples in the metric display coordinate frame and rendered as an overlay on the final map. Start and end positions are marked with distinct colours for operational awareness.

3.7.3 Coverage Area Estimation

The total coverage area is estimated by rasterizing all frame footprints onto a union mask at approximately 1 m/px resolution:

$$A_{\text{coverage}} = N_{\text{pixels}} \times r^2, \quad (3.18)$$

where N_{pixels} is the count of non-zero pixels in the union mask and r is the raster resolution.

3.7.4 Seam SSIM

To quantify blending quality at tile boundaries, the system computes the Structural Similarity Index (SSIM) [16] between adjacent tile edge strips. This metric captures luminance continuity across seams and serves as a proxy for visual coherence. Higher SSIM values indicate better alignment and blending at boundaries.

Chapter 4

Pose Graph Optimization and View-Aware Compositing

This chapter presents the two primary algorithmic contributions of this thesis. Section 4.1 describes the translation-only pose graph that refines GPS positions using visual constraints. Section 4.2 describes the view-aware compositor that suppresses directional artefacts through heading-conflict arbitration.

4.1 Translation-Only Pose Graph

Consumer GNSS provides a useful placement prior but often introduces metre-level drift that is readily visible in an incremental mosaic. To improve geometric consistency without resorting to full bundle adjustment, the method estimates only a two-dimensional translation correction for each frame. The key design challenge is that naive pairwise image registration can inject contradictory constraints in the presence of relief displacement and oblique viewing. The implemented graph therefore combines pairwise visual alignment with explicit compatibility and rigidity checks.

4.1.1 Problem Formulation

Let $\mathbf{c}_i = [c_i^x, c_i^y]^T$ denote the unknown horizontal correction for frame i , and let $\mathbf{g}_i$ denote the GPS-derived position. The method solves:

$$\min_{\{\mathbf{c}_i\}} \sum_i w_{\text{gps}} \|\mathbf{c}_i\|_2^2 + \sum_{(i,j) \in \mathcal{E}} w_{\text{match}} \left\| (\mathbf{c}_j - \mathbf{c}_i) + \hat{\Delta}_{ij} \right\|_2^2, \quad (4.1)$$

where $\mathcal{E}$ is the set of accepted frame pairs, and $\hat{\Delta}_{ij}$ is the pairwise displacement cue derived from feature matching. The first term keeps the solution close to the GPS prior (zero correction), and the second term penalises residual disagreement between matched frame pairs.

This formulation yields a sparse linear least-squares system. By restricting the optimization to two-dimensional translations and treating gimbal rotations as fixed, the problem

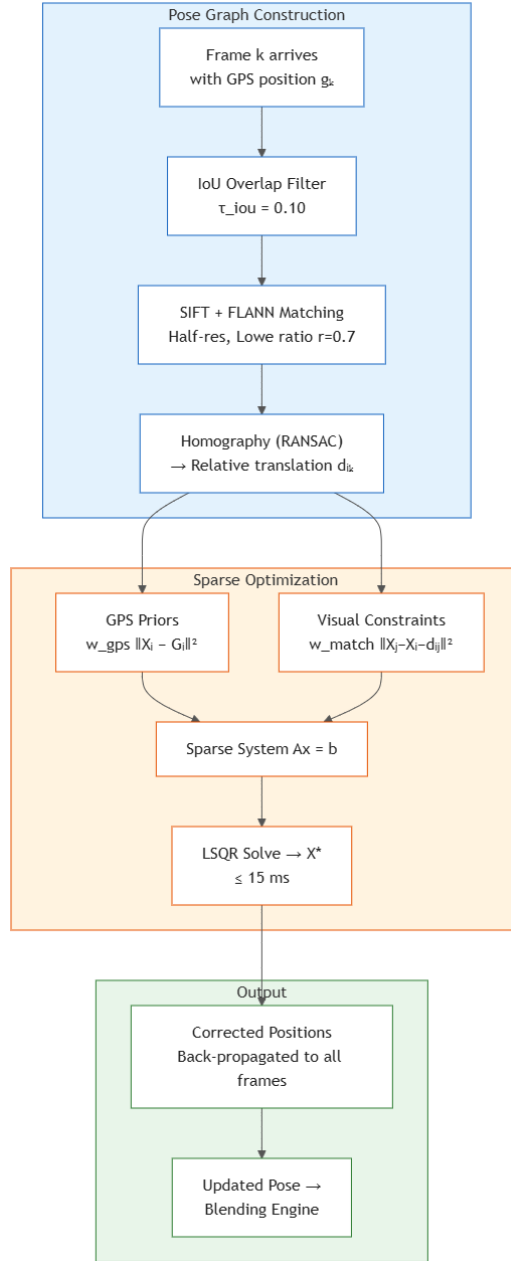


FIGURE 4.1: Pose graph optimization pipeline: IoU-filtered SIFT matching builds visual constraints, which are jointly solved with GPS priors via sparse LSQR.

is *convex* - it has a unique global minimum that can be found exactly, without the iterative nonlinear optimization required by full 6-DoF pose graphs.

4.1.2 Candidate Pair Selection

When a new frame i arrives, it is compared with two classes of earlier frames:

- **Recent frames:** The k most recent frames (default $k = 6$) within a sliding temporal window. These capture sequential overlap.

- **Spatial candidates:** Older frames whose projected ground footprints overlap the current footprint above a prescribed IoU threshold (default $\tau_{\text{IoU}} = 0.10$). Footprints are obtained by projecting the four image corners to the ground plane using the initial pose. A spatial index with configurable cell size (default 250 m) provides $O(1)$ candidate lookup. To avoid quadratic growth, at most k_{spatial} (default 3) non-sequential candidates are retained per frame, selected by highest IoU.

4.1.3 View Compatibility Filtering

Candidate pairs are further filtered by a view-compatibility test that rejects edges likely dominated by view-induced parallax. For each frame, the camera optical axis in world coordinates is:

$$\mathbf{v}_i = \mathbf{R}_i^0 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. \quad (4.2)$$

From this vector, the off-nadir angle and horizontal viewing direction are computed:

$$\alpha_i = \arctan\left(\frac{\sqrt{v_{i,x}^2 + v_{i,y}^2}}{\max(v_{i,z}, \epsilon)}\right), \quad \psi_i = \text{atan2}(v_{i,y}, v_{i,x}), \quad (4.3)$$

where ϵ is a small constant for numerical stability.

Pairs are rejected if: (a) their off-nadir angles differ by more than 20° , indicating incompatible viewing geometry; or (b) both views are sufficiently oblique (off-nadir $> 18^\circ$) and their horizontal headings differ by more than 25° .

4.1.4 Ground-Plane Pairwise Constraints

For each accepted pair (i, j) , SIFT keypoints are extracted from downsampled grayscale images and matched using FLANN with Lowe's ratio test [15, 17]. Let $\mathbf{u}_{i,k} = [u_{i,k}, v_{i,k}, 1]^\top$ be a matched keypoint in frame i . Its camera ray is:

$$\mathbf{r}_{i,k}^{\text{cam}} = \mathbf{K}^{-1} \mathbf{u}_{i,k}, \quad (4.4)$$

which is normalized and transformed to the world frame:

$$\mathbf{r}_{i,k}^{\text{world}} = \mathbf{R}_i^0 \frac{\mathbf{r}_{i,k}^{\text{cam}}}{\|\mathbf{r}_{i,k}^{\text{cam}}\|_2}. \quad (4.5)$$

The corresponding ground-plane intersection is obtained by ray casting to $Z = 0$:

$$\lambda_{i,k} = -\frac{t_{i,z}}{r_{i,k,z}^{\text{world}}}, \quad \mathbf{P}_{i,k} = \mathbf{t}_i^{\text{gps}} + \lambda_{i,k} \mathbf{r}_{i,k}^{\text{world}}. \quad (4.6)$$

The same operation is applied to the matching point in frame j . Under perfect placement and an exactly planar scene, corresponding points would map to the same ground

coordinate. The observed disagreement is:

$$\Delta_{ij,k} = \mathbf{P}_{j,k}^{\text{xy}} - \mathbf{P}_{i,k}^{\text{xy}}, \quad (4.7)$$

where only the horizontal components are retained.

A robust pairwise translation cue is initialized by the component-wise median of these displacements:

$$\tilde{\Delta}_{ij} = \text{median}_k(\Delta_{ij,k}). \quad (4.8)$$

Matches whose residual to this median exceeds a fixed tolerance (2 m) are rejected, and the final pairwise cue is recomputed from the surviving inliers:

$$\hat{\Delta}_{ij} = \frac{1}{|\mathcal{I}_{ij}|} \sum_{k \in \mathcal{I}_{ij}} \Delta_{ij,k}, \quad (4.9)$$

where $\mathcal{I}_{ij}$ is the accepted inlier set.

4.1.5 Parallax Screening by Spatial Rigidity

The planar assumption is most likely to fail around tall structures and at image peripheries. To reduce the influence of such effects, the method applies two heuristics:

Centre preference. For a keypoint location $\mathbf{q} = [u, v]^T$, a normalized radius is computed:

$$\rho(\mathbf{q}) = \frac{\|\mathbf{q} - \mathbf{q}_c\|_2}{\sqrt{(W/2)^2 + (H/2)^2}}, \quad (4.10)$$

where $\mathbf{q}_c$ is the image centre. If enough correspondences exist within a central radius threshold ($\rho \leq 0.72$, default), only this subset is retained. Central matches are less affected by relief displacement because they are closer to nadir.

Rigidity check. The pair is checked for spatial rigidity by comparing the median displacement estimated from central matches ($\rho \leq 0.35$) and near-edge matches ($\rho \geq 0.58$):

$$\left\| \tilde{\Delta}_{ij}^{\text{center}} - \tilde{\Delta}_{ij}^{\text{edge}} \right\|_2 \leq \tau_{\text{rigid}}. \quad (4.11)$$

If this condition is violated (default $\tau_{\text{rigid}} = 2.5$ m), the pair is rejected. This is not a full parallax model, but a practical safeguard against grossly inconsistent constraints under relief displacement.

4.1.6 Sparse Translation-Only Optimization

Equation (4.1) is expanded into a sparse linear system $\mathbf{Ax} = \mathbf{b}$. The variable vector is $\mathbf{x} = [c_0^x, c_0^y, c_1^x, c_1^y, \dots, c_n^x, c_n^y]^T$, containing $2n$ unknowns for n frames.

The coefficient matrix $\mathbf{A}$ has $(n + |\mathcal{E}|) \times 2$ rows (GPS priors plus edge constraints, each contributing two rows for x and y components):

- **GPS prior rows:** For each frame i : $\sqrt{w_{\text{gps}}} \cdot c_i^x = 0$ and $\sqrt{w_{\text{gps}}} \cdot c_i^y = 0$. Each row has one non-zero entry.
- **Edge constraint rows:** For each edge (i, j) : $\sqrt{w_{\text{match}}} \cdot (c_j - c_i) = \sqrt{w_{\text{match}}} \cdot (-\hat{\Delta}_{ij})$. Each row has two non-zero entries.

The matrix is constructed in LIL (List of Lists) sparse format for efficient incremental construction, then converted to CSR format for the solve. The system is solved using the **LSQR** algorithm [18] from SciPy’s sparse linear algebra module, which is based on the Golub-Kahan bidiagonalization procedure. LSQR is preferred over direct factorization because it handles rectangular systems, exploits sparsity without extra fill-in, and converges in $O(n)$ iterations for well-conditioned problems.

The corrected pose of frame i becomes:

$$\mathbf{T}_i = \begin{bmatrix} \mathbf{R}_i^0 & \mathbf{t}_i^{\text{gps}} + \begin{bmatrix} c_i^x \\ c_i^y \\ c_i^z \\ 0 \end{bmatrix} \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (4.12)$$

4.1.7 Solve Strategy

The system re-solves whenever new valid edges are added, subject to a throttling policy:

- During the first 10 frames (bootstrap phase), the system solves after every new edge to establish the graph quickly.
- After bootstrap, re-solves occur every N frames (default $N = 5$) to keep runtime bounded in longer sequences.
- A manual `flush()` method forces a final solve at the end of a sequence.

Each re-solve updates *all* frame corrections, not just the latest one. This backward propagation provides a simple form of global consistency, analogous to loop closure in SLAM, without requiring explicit loop detection.

4.1.8 Feature Detector Choice: SIFT vs. ORB

The system supports both SIFT [17] and ORB [19] as feature detectors. SIFT provides scale-invariant features with high matching quality but requires significantly more computation. ORB is a binary descriptor that can be computed and matched much faster.

In the translation-only formulation, the feature detector’s role is limited to providing pairwise displacement cues - the geometric precision requirements are lower than in full SfM, where sub-pixel reprojection accuracy is needed. This suggests that ORB may provide sufficient constraint quality at much lower computational cost, a hypothesis validated experimentally in Chapter 6.

4.2 View-Aware Incremental Compositing

After pose refinement, each image is projected to the ground plane and inserted into the global tiled mosaic. The update rule extends the basic max-weight arbitration described in Chapter 3 with heading-conflict detection and nadir-preference scoring.

4.2.1 View Metrics

For each frame, the optical axis vector $\mathbf{v}_i$ is analysed to compute its off-nadir angle α_i and heading ψ_i . A frame-level nadir preference is defined as:

$$S_i^{\text{nadir}} = \exp\left(-\left(\frac{\alpha_i}{\sigma}\right)^2\right), \quad (4.13)$$

where $\sigma = 35^\circ$ controls the rate at which oblique views are penalized. This score is clipped to $[0.05, 1.0]$ to avoid degenerate weights.

A spatial confidence mask $W_i(\mathbf{x})$ is computed over the image domain using the centre-weighted function from Section 3.6. After warping, the local pixel score becomes:

$$S_i(\mathbf{x}) = W_i(\mathbf{x}) \cdot S_i^{\text{nadir}}. \quad (4.14)$$

Thus, incoming observations are favoured when they are both centrally observed and close to nadir.

4.2.2 Heading-Conflict Detection

Let $S_{\text{old}}(\mathbf{x})$ be the stored score at a map location and $S_{\text{new}}(\mathbf{x})$ the score of the incoming warped image. Let ψ_{old} and ψ_{new} denote the stored and incoming headings. A *heading conflict* is declared when the wrapped angular difference exceeds a threshold:

$$|((\psi_{\text{old}} - \psi_{\text{new}} + 180^\circ) \bmod 360^\circ) - 180^\circ| \geq \tau_{\text{heading}}, \quad (4.15)$$

where $\tau_{\text{heading}} = 35^\circ$ (default).

4.2.3 Overlap Arbitration Rules

The update rule is:

1. **Empty pixel:** Insert the new observation unconditionally.
2. **Occupied, no heading conflict:** Replace the old observation only if the incoming score exceeds the stored score ($S_{\text{new}} > S_{\text{old}}$). This is the standard max-weight rule.
3. **Heading conflict:** Replace the old observation only when the incoming view is sufficiently more nadir-facing (nadir score exceeds stored nadir score plus margin $\delta = 0.12$) *or* its total score substantially exceeds the stored score ($S_{\text{new}} > 1.20 \cdot S_{\text{old}}$).

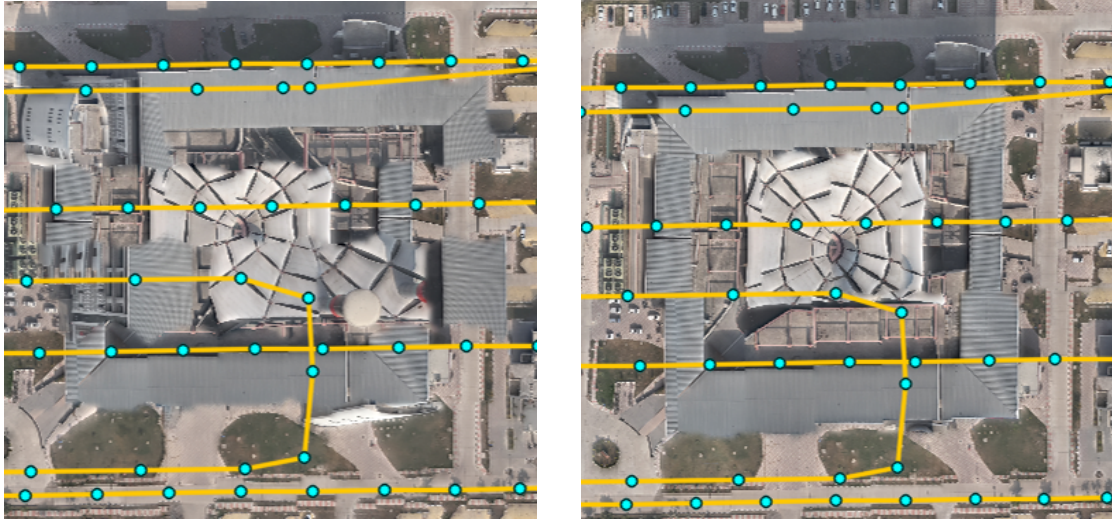
The elevated replacement threshold in the conflict case prevents oscillation between opposing views. The nadir-preference override ensures that near-nadir views take precedence over oblique views even when their centre-weighted scores are similar.

4.2.4 Rationale and Limitations

This policy is designed to reduce double-imaging artefacts when elevated structures are viewed from opposing directions. A building viewed from the north appears to lean south, while the same building viewed from the south appears to lean north. Without heading-conflict awareness, overlapping these views produces a double-façade effect.

The view-aware compositor mitigates this by preferring the more nadir-facing observation in contested regions. This is a practical heuristic rather than a physically complete solution: it does not model occlusion or reconstruct the 3D structure. It is effective when one flight direction is substantially more nadir-facing than the other, which is the common case in survey patterns with gimbal stabilization.

The conflict statistics (number of detected conflict pixels, number of overrides) are tracked and reported as part of the evaluation framework.



(a) Raw placement / no refinement

(b) Proposed refinement + arbitration

FIGURE 4.2: Relief-displacement artifacts before and after the proposed translation refinement and view-aware arbitration.

Chapter 5

Applications and Deployment

This chapter describes the two application systems built around the core mapping engine and the deployment pipeline that bridges drone hardware to the software stack. Section 5.1 presents the web-based Drone Mapper application. Section 5.2 describes the SkyForge Ground Control Station. Section 5.3 details the MAVLink integration pipeline. Section 5.4 discusses the real-world field deployment.

5.1 Drone Mapper: Web-Based Mapping Application

The first application built around the core engine is a web-based mapping tool implemented using Streamlit¹, providing a browser-accessible interface for drone image processing and map generation. The source code is publicly available at <https://github.com/lyrnox/drone-mapper>.

5.1.1 Interface Design

The application features a right-side sidebar (achieved via CSS `flex-direction: row-reverse`) with the map prominently displayed in the left and centre portions of the screen, following GIS application conventions. The sidebar provides:

- **Camera parameters:** Focal length, sensor dimensions, and image resolution.
- **Map settings:** Output resolution (m/px), number of Laplacian bands, tile size.
- **Optimizer controls:** GPS weight (w_{gps}), match weight (w_{match}), IoU threshold, minimum matches, and solve interval.
- **Mode selection:** Raw GPS, Pose Graph Optimization, or A/B Comparison.

¹<https://streamlit.io/>

5.1.2 A/B Comparison Framework

The A/B comparison mode instantiates two independent **MultiBandMap2D** engines: one processing raw GPS poses and one processing optimized poses from the pose graph. Both process the same input frames simultaneously, enabling:

- Side-by-side visual comparison of map quality.
- Seam SSIM measurement at tile boundaries for both pipelines.
- Per-frame correction magnitude tracking ($\|\mathbf{x}_i^{\text{opt}} - \mathbf{g}_i\|$).
- Flight path displacement visualization with correction arrows.

5.1.3 Cloud Deployment

The system was deployed on a DigitalOcean Ubuntu droplet with the following infrastructure:

- **Nginx reverse proxy** with WebSocket upgrade support and `client_max_body_size 200M` for large DJI image uploads.
- **Systemd service** (`drone-mapper.service`) for automatic process management, restart-on-failure, and boot integration.
- **Automated deployment:** GitHub webhook triggers `git pull` → `pip install` → `systemctl restart`.
- **Image downscaling:** Images exceeding 2000 px on their longer side are down-scaled proportionally, with camera intrinsics adjusted accordingly:

$$\mathbf{K}' = \begin{bmatrix} f_x \cdot s & 0 & c_x \cdot s \\ 0 & f_y \cdot s & c_y \cdot s \\ 0 & 0 & 1 \end{bmatrix}, \quad s = \frac{\text{MAX_DIM}}{\max(W, H)}. \quad (5.1)$$

5.2 SkyForge Ground Control Station

The second and more comprehensive application is a professional desktop Ground Control Station which we have named **SkyForge GCS**, implemented as a 3100-line PyQt6 application. SkyForge was developed to address the limitations of the web-based tool for field deployment scenarios requiring low-latency processing, live video feeds, and offline operation.

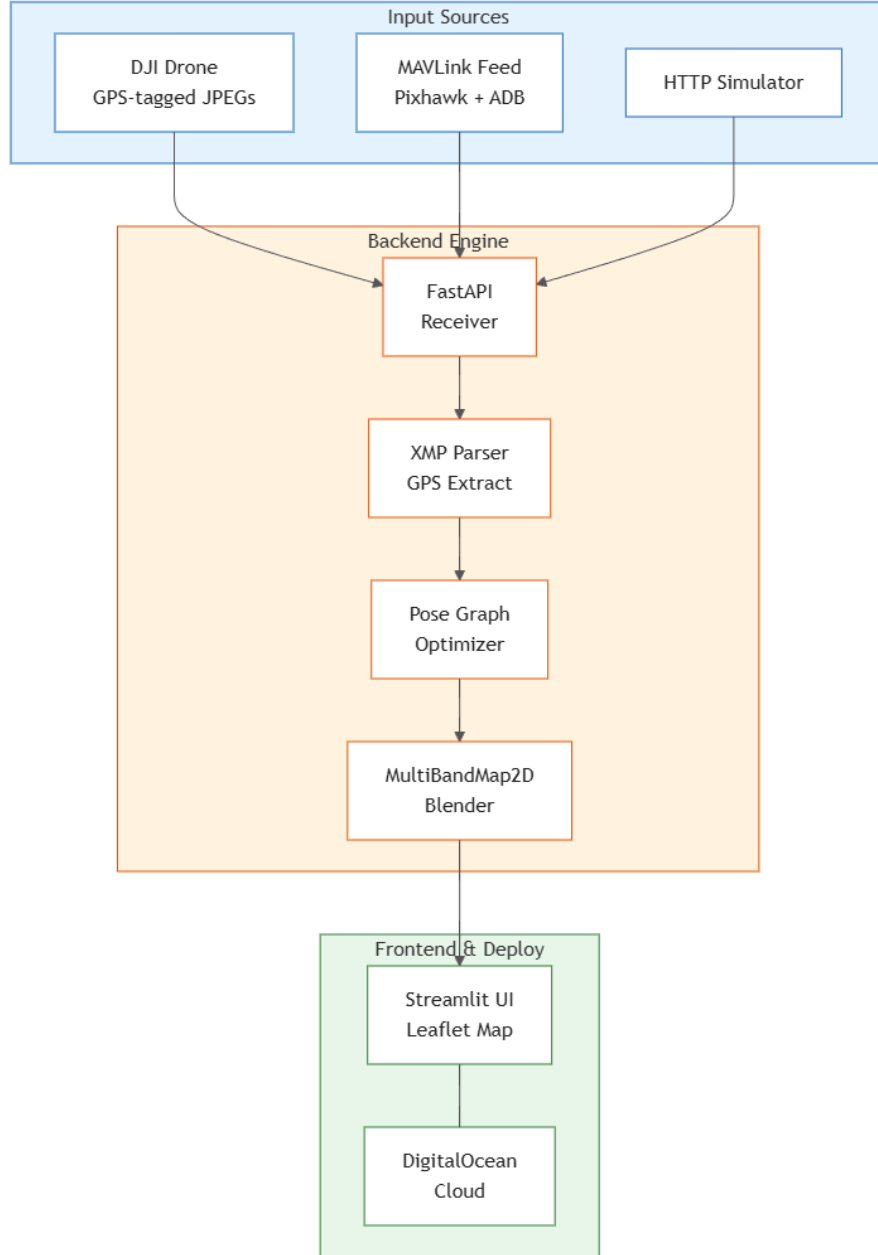


FIGURE 5.1: Extended system architecture: Pose Graph Optimizer, Streamlit UI, cloud deployment, and MAVLink live feed pipeline.

5.2.1 Architecture

SkyForge employs a multi-threaded architecture with three primary worker threads:

- **MapProcessorWorker:** Background thread that dequeues images from a thread-safe queue, processes them through the **MapperService**, and emits processed results and rendered maps via Qt signals. For MAVLink sources, the queue implements a “latest-only” policy: incoming live frames replace any pending live frames to prevent queue buildup.

- **SimulationWorker:** Feeds images from a local folder at a configurable rate, supporting both manifest-based (JSONL) and alphabetically sorted playback. This enables offline testing and replay of recorded flight data.
- **MAVLinkWorker:** Bridges MAVLink telemetry and camera feeds into the processing pipeline. Implements auto-retry connection logic (5 attempts with 3-second delays), heartbeat watchdog (10-second timeout), and telemetry-to-video time synchronization.

5.2.2 User Interface

The application features a professional dark-themed interface inspired by mission-critical GCS software:

- **Central map view:** Zoomable, pannable display of the evolving orthomosaic with satellite underlay support. The map view supports multiple rendering modes including drone imagery only, satellite background only, and composited overlay.
- **Telemetry HUD:** Real-time heads-up display showing GPS coordinates, altitude, heading, speed, and gimbal angles in a monospace font for readability.
- **Live camera feed:** Direct display of the drone's camera stream with frame quality indicators.
- **Performance metrics:** Processing time, render time, queue depth, and throughput statistics updated in real time.
- **Theme support:** Switchable dark/light themes with comprehensive stylesheet coverage for all Qt widgets.

5.2.3 Video Frame Intelligence

A significant engineering challenge in live drone mapping is distinguishing genuine camera frames from decoder artefacts. SkyForge implements a multi-stage frame validation pipeline:

1. **Blank frame detection:** Flat gray decoder placeholder frames from unstable RTSP streams are detected by checking grayscale standard deviation (< 6.0) and HSV saturation mean (< 8.0), or sharpness via Laplacian variance (< 18.0).
2. **Frozen frame detection:** Consecutive frames with identical content but moving GPS positions are detected by computing a low-resolution signature (160×90 grayscale thumbnail) and comparing mean absolute difference (< 1.5).
3. **Buffered frame detection:** Stale frames from video buffers are detected using phase correlation to measure sub-pixel motion (< 1.0 px) against telemetry-indicated movement (> 1.0 m).
4. **Altitude gate:** Frames below a minimum altitude threshold (2.0 m AGL) are skipped to avoid processing ground-level imagery.

5.2.4 Satellite Map Integration

SkyForge supports satellite map tile underlays for contextual reference. The `SatelliteTileProvider` in the backend fetches and caches map tiles from standard tile servers, compositing them behind the drone imagery in the rendered output. The GPS origin coordinates from the pose extractor are used to compute the correct tile coordinates at the appropriate zoom level.

5.2.5 Configuration and Persistence

Application settings are managed through a JSON configuration file (`skyforge_config.json`), with defaults for:

- Camera profiles (DJI Mavic 3T, Phantom 3, custom)
- MAVLink connection strings
- Map resolution and optimizer parameters
- Video source and capture settings
- Window geometry and dock positions (saved via QSettings)

5.3 MAVLink Deployment Pipeline

A critical gap between the research prototype and a field-ready system was the absence of a live data acquisition method. The earlier evaluation relied entirely on pre-recorded DJI images with embedded XMP metadata. To enable real-time mapping from non-DJI platforms, a complete deployment pipeline was built that bridges Pixhawk-class flight controllers to the mapping engine via the MAVLink protocol [20].

5.3.1 Hardware Chain

The physical data path is: **Pixhawk autopilot** → **Skydroid Air Unit** (serial) → **MK15 ground controller** (2.4 GHz link) → **USB-C ADB forwarding** → **host PC**.

5.3.2 Telemetry Acquisition

The `MAVLinkTelemetry` class implements a background reader thread that automatically discovers the MAVLink connection by probing a list of UDP and TCP endpoints (e.g., `udp:0.0.0.0:14550`, `tcp:127.0.0.1:5760`) until a heartbeat is received. Once connected, it continuously parses four MAVLink message types:

- `GLOBAL_POSITION_INT`: GPS coordinates, absolute and relative altitude, velocity vector.

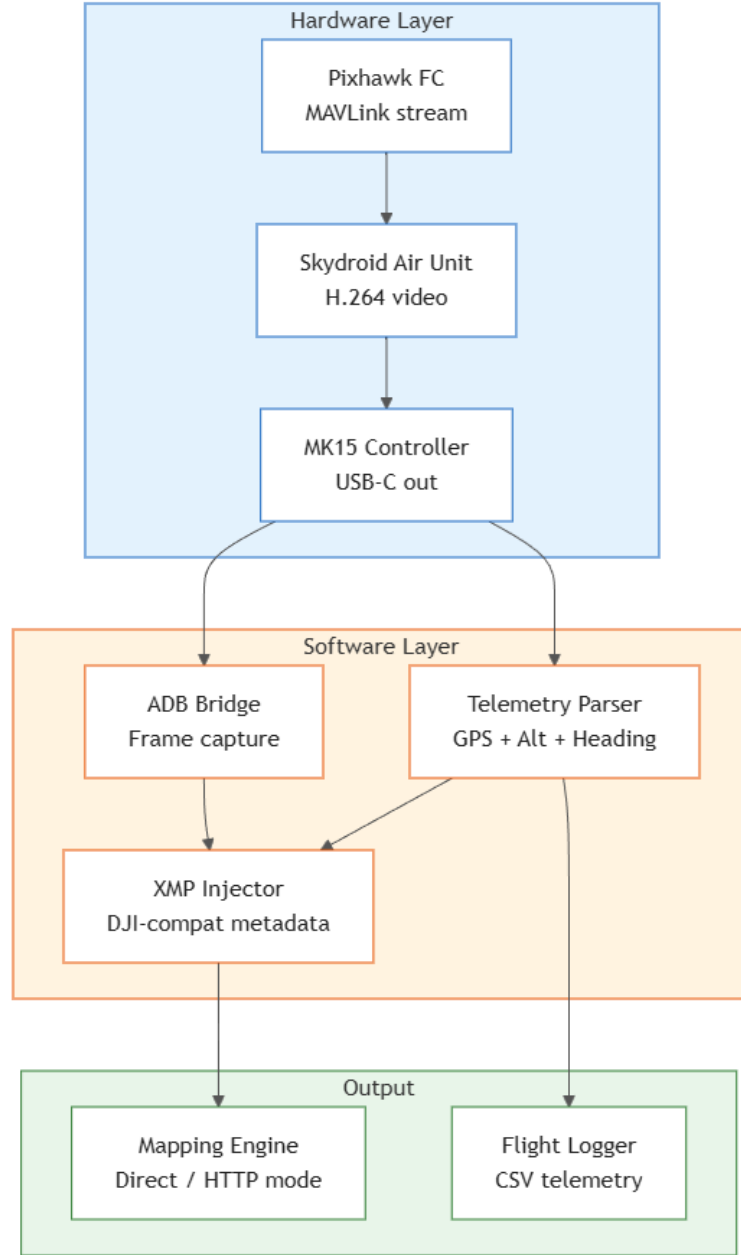


FIGURE 5.2: MAVLink deployment pipeline data flow: Pixhawk → Skydroid Air Unit → MK15 → ADB → XMP injection → mapping engine.

- **ATTITUDE:** Aircraft body roll, pitch, and yaw (radians → degrees).
- **MOUNT_ORIENTATION:** Gimbal Euler angles in degrees (MAVLink v2).
- **GIMBAL_DEVICE_ATTITUDE_STATUS:** Gimbal orientation as a quaternion, converted to Euler angles via SciPy's `Rotation.from_quat()`.

All telemetry fields are stored in a thread-safe dictionary and exposed through a `snapshot()` method that returns an atomic copy of the latest state. A GPS quality gate requires 3D fix and $\text{HDOP} \leq 5.0$ before frames are accepted.

5.3.3 DJI-Compatible XMP Injection

Rather than modifying the mapping engine’s metadata parser for each new drone platform, the pipeline constructs DJI-compatible XMP XML from MAVLink telemetry and injects it into the JPEG APP1 segment. This approach makes any MAVLink-capable airframe appear identical to a DJI drone at the metadata level, allowing the entire downstream processing stack to be reused without modification.

Gimbal angles use the real gimbal orientation when available; otherwise, the pipeline defaults to a fixed nadir assumption (roll = 0° , pitch = -90° , yaw = aircraft heading).

5.3.4 Camera Capture

The `CameraCapture` class wraps OpenCV’s `VideoCapture` with support for both local HDMI capture cards (by device index) and RTSP streams from the Skydroid MK15 (`rtsp://192.168.144.108:554/stream=1`). For RTSP, the FFMPEG backend with UDP transport is used for reduced latency. An auto-reconnection mechanism retries up to five times on consecutive frame grab failures.

5.3.5 Operational Modes

Two feed modes are provided:

1. **HTTP mode** (`-live`): Geotagged JPEGs are POSTed to the FastAPI `/feed-image` endpoint, suitable for distributed setups where the mapper runs on a separate machine.
2. **Direct mode** (`-direct`): The `MapperService.process_image()` method is called in-process, eliminating HTTP serialization overhead. An optional live OpenCV window shows the evolving orthomosaic with flight path overlay.

5.3.6 Flight Data Logging

The `FlightLogger` class creates timestamped session directories containing: (a) the complete telemetry snapshot for each captured frame in JSON Lines format, (b) the geotagged JPEG for every frame, and (c) session metadata including camera source, mode, interval, and connection string. This logging capability enables offline re-processing and algorithm development using real flight data.

5.4 Field Deployment

The SkyForge GCS application was deployed for field trials with the Indian Army. The deployment validated the complete hardware-software chain from Pixhawk-based airframes through the MAVLink telemetry pipeline to the real-time mapping engine. Key outcomes include:



(a) Showing the software setup and usage to operatives at the lab



(b) Flying the drone at the field

FIGURE 5.3: Preparation and field deployment of the SkyForge GCS with the Indian Army.



(a) The drone flying during the deployment



(b) Checking the real-time mapped screen on the SkyForge GCS

FIGURE 5.4: Real-time mapping in progress during field trials.

- **Operational validation:** The system successfully generated orthomosaics in real time during live drone flights over operational terrain.
- **Robustness:** The auto-reconnection mechanisms in both the MAVLink telemetry and camera capture subsystems handled the intermittent connectivity typical of field wireless links.
- **Usability feedback:** The deployment identified the need for the frame intelligence pipeline (Section 5.2) to handle the decoder artefacts and buffer latency characteristic of RTSP streams over wireless video links.

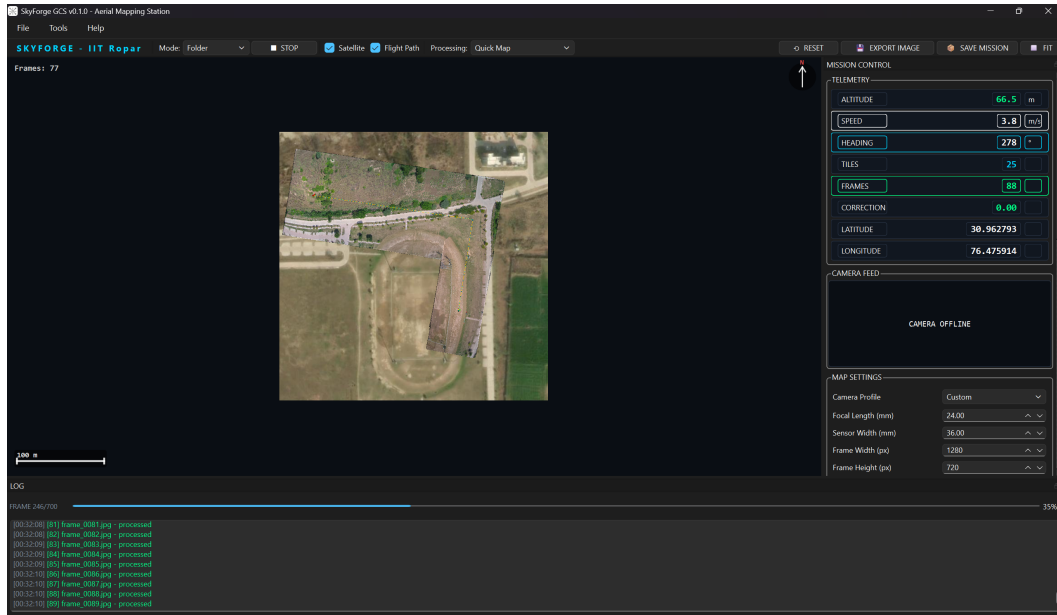


FIGURE 5.5: SkyForge UI during operation, displaying the generated orthomosaic over a satellite underlay.

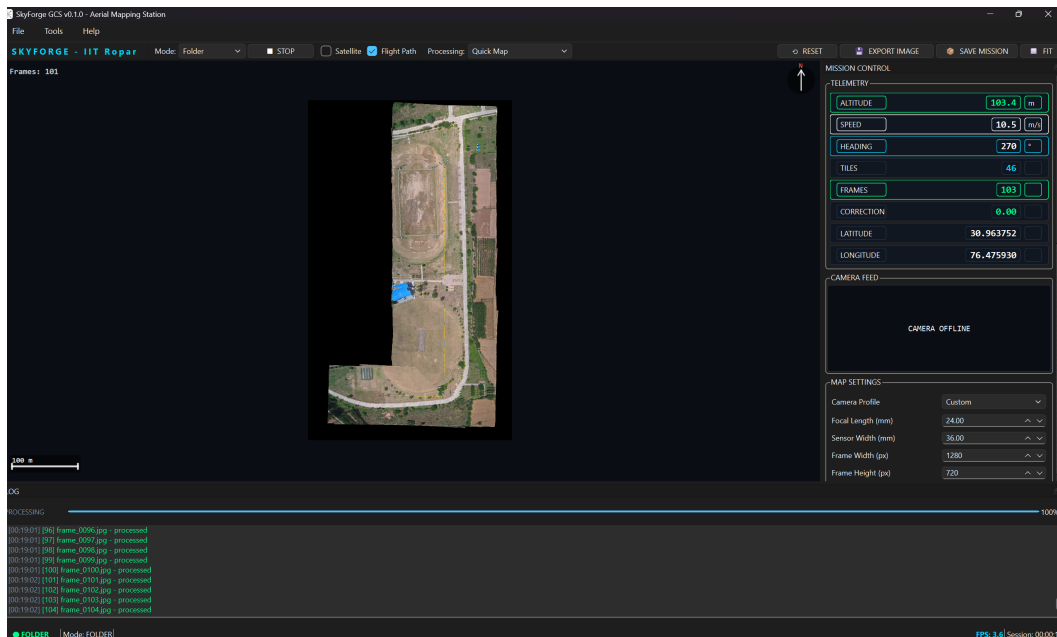


FIGURE 5.6: SkyForge UI during operation without the satellite underlay, isolating the drone-generated map.

- **Performance:** Real-time mapping was sustained on laptop hardware (CPU-only) with the optimizer enabled, confirming the feasibility of field deployment without GPU infrastructure.

The field deployment experience directly influenced several engineering decisions in the GCS, including the telemetry-to-video time synchronization mechanism, the altitude-based frame gate, and the configurable video delay compensation.

Chapter 6

Experimental Evaluation and Conclusion

This chapter presents the experimental evaluation of the proposed system, followed by a discussion of results, limitations, and future directions.

6.1 Datasets and Experimental Setup

Three flight sequences of increasing complexity are used, summarized in Table 6.1.

data1 covers a stretch of the Sutlej river and its suburban surroundings. The 241 images are captured at 4000×3000 px from approximately 142 m AGL with a 2 s inter-frame interval.

data2 is the PhantomVillage sequence from the Map2DFusion repository [4], covering a dense village scene. It provides 406 frames at 1920×1080 px with an externally supplied trajectory file.

data3 covers the IIT Ropar campus, acquired with the same drone as data1. With 140 frames from approximately 120 m AGL, it features buildings, roads, and vegetation.

All experiments were executed single-threaded on an AMD Ryzen 7 7840HS processor (8 cores, 16 threads, 16 GB RAM). The map resolution is fixed at 0.5 m/px with a tile size of 512 px and two Laplacian bands. The optimizer uses $w_{\text{gps}} = 9.0$, $w_{\text{match}} = 0.25$, a solve interval of every 5 frames, a maximum of 700 features per frame, and a spatial cell size of 250 m.

We compare three processing variants: (i) *Raw GPS*, which projects each frame using the original GNSS/gimbal pose without correction; (ii) *Opt. SIFT*, which adds the translation pose graph using SIFT features; and (iii) *Opt. ORB*, which replaces SIFT with ORB in an otherwise identical pipeline. All three use the view-aware compositor.

6.2 Runtime Performance

Table 6.2 reports per-frame latency statistics on clean trajectories.

TABLE 6.1: Dataset characteristics.

Property	data1	data2	data3
Scene	River / suburban	Village	University campus
Frames	241	406	140
Image resolution (px)	4000×3000	1920×1080	4000×3000
Pose source	DJI EXIF	External traj.	DJI EXIF
Mean altitude (m)	142.2	160.9	120.2
Capture interval (s)	2.0	1.6	2.3
Coverage (m^2)	727 335	723 534	523 278
Total input (MB)	1 397	252	743

TABLE 6.2: Per-frame runtime on clean trajectories (AMD Ryzen 7 7840HS, 0.5 m/px).

Dataset	Variant	Avg total (ms)	P95 total (ms)	Avg map (ms)	Avg opt (ms)	FPS	Miss rate (%)
data1	Raw GPS	96	108	16	-	10.4	0
	Opt. SIFT	569	683	24	407	1.8	0
	Opt. ORB	188	215	21	44	5.3	0
data2	Raw GPS	53	60	26	-	18.8	0
	Opt. SIFT	441	562	33	378	2.3	1.0
	Opt. ORB	137	216	31	80	7.3	0
data3	Raw GPS	147	214	40	-	6.8	0
	Opt. SIFT	555	791	51	394	1.8	0
	Opt. ORB	232	435	49	76	4.3	0

Raw GPS processing is the fastest variant across all datasets: 96 ms mean on data1, 53 ms on data2, and 147 ms on data3. The ORB optimizer achieves a $4.7\times$ to $9.3\times$ speedup over SIFT in the optimization stage, keeping total latency at 137–232 ms (4.3–7.3 FPS). The deadline miss rate is 0% on all clean runs except data2 with SIFT (1.0%).

6.3 View-Aware Blending Evaluation

Table 6.3 isolates the effect of the view-aware compositor.

The most informative case is data2 (village), where the view-aware compositor detects 2 837 423 conflict pixels and overrides 278 658 of them. Visual inspection confirms suppression of double-façade artefacts near buildings, an effect not captured by the tile-boundary SSIM metric.

6.4 Mosaic Quality Analysis

Table 6.4 reports mosaic quality metrics on clean trajectories.

TABLE 6.3: Effect of view-aware overlap arbitration on blending quality.

Dataset	Compositor	Seam SSIM	Map (ms)	Conflict pixels	Conflict overrides
data1	Basic	0.9575	25.1	0	0
	View-aware	0.9575	16.1	0	0
data2	Basic	0.9094	25.9	0	0
	View-aware	0.9094	26.5	2 837 423	278 658
data3	Basic	0.9356	40.4	0	0
	View-aware	0.9361	38.2	0	0



FIGURE 6.1: A/B comparison on IIT Ropar flights: (left) raw GPS with visible tile-boundary seams, (right) pose-graph-optimized with improved alignment.

The optimizer variants do not degrade seam quality relative to raw GPS. On data2, the optimized variants improve SSIM from 0.909 to 0.913–0.914, indicating reduced seam misalignment. Memory usage is identical across variants because the tile grid is determined by coverage area.

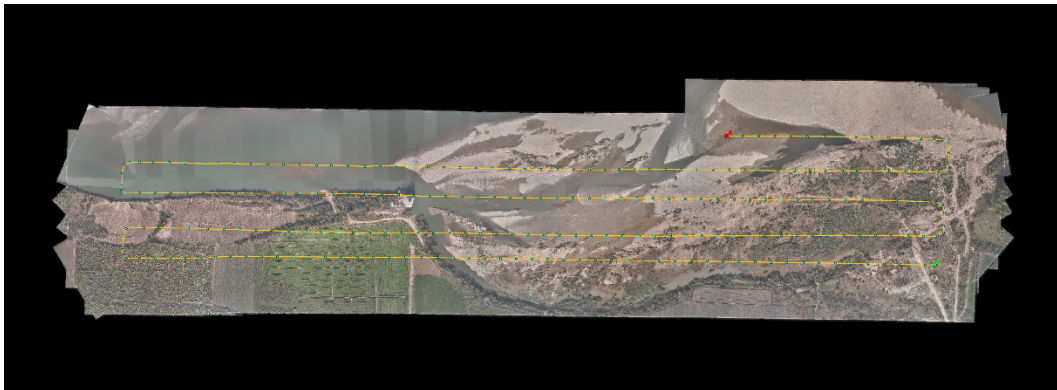


FIGURE 6.2: Orthomosaic from DJI Mavic 3T flights over Ropar area, with optimized flight path overlay.

TABLE 6.4: Mosaic quality metrics on clean trajectories.

Dataset	Variant	Seam SSIM	Avg IoU	Coverage (m ²)	Mem. (MB)
data1	Raw GPS	0.958	0.688	727 335	189
	Opt. SIFT	0.957	0.688	727 396	189
	Opt. ORB	0.957	0.688	727 392	189
data2	Raw GPS	0.909	0.768	723 534	150
	Opt. SIFT	0.913	0.768	723 258	150
	Opt. ORB	0.914	0.768	723 354	150
data3	Raw GPS	0.936	0.720	523 278	150
	Opt. SIFT	0.937	0.720	523 277	150
	Opt. ORB	0.937	0.720	523 284	150

6.5 Pose-Graph Analysis

Table 6.5 reports pose-graph statistics on clean trajectories.

TABLE 6.5: Pose-graph statistics on clean trajectories.

Dataset	Detector	Edges	Res. before	Res. after	Δ %	Avg corr (m)
data1	SIFT	1 433	4.60	4.26	−7.3	0.088
	ORB	1 333	4.42	4.05	−8.3	0.079
data2	SIFT	3 130	6.73	6.47	−3.9	0.208
	ORB	2 632	5.57	5.34	−4.1	0.148
data3	SIFT	566	5.82	5.49	−5.7	0.111
	ORB	490	5.40	5.06	−6.4	0.090

SIFT generates more edges than ORB (e.g., 1 433 vs. 1 333 on data1), but both detectors achieve similar relative residual reductions (3.9–8.3%). Mean corrections on clean trajectories are small (0.079–0.208 m), confirming the optimizer does not over-correct with good GPS.

6.6 Robustness Under Synthetic GPS Drift

6.6.1 Fixed-Bias Stress Test

Table 6.6 presents a 3 m linear bias experiment on data1.

TABLE 6.6: Synthetic GPS bias experiment on data1 (3 m linear bias).

Variant	Raw err (m)	Eval err (m)	Corr (m)	Res. before	Res. after	Seam SSIM
Raw GPS	1.589	1.589	-	-	-	0.957
Opt. SIFT	1.589	1.649	0.082	4.42	4.07	0.957
Opt. ORB	1.589	1.644	0.073	4.26	3.87	0.957

6.6.2 Multi-Drift Robustness

Table 6.7 extends the analysis across five drift profiles.

TABLE 6.7: Robustness under varying synthetic drift profiles (data1, ORB optimizer).

Drift profile	Mag. (m)	Raw err	Eval err	Res. bef.	Res. aft.	Seam SSIM
Clean	0	0.000	0.079	4.42	4.05	0.957
Gauss 1 m	1	1.167	1.139	4.68	4.20	0.957
Gauss 2 m	2	2.334	2.268	5.42	4.62	0.956
RW 2 m	2	1.081	1.138	4.52	4.09	0.957
RW 4 m	4	2.162	2.211	4.62	4.14	0.955
Bias 3 m	3	1.589	1.644	4.26	3.87	0.957

Seam SSIM remains within 0.955–0.957 across all drift profiles, demonstrating that the pipeline maintains visual quality under substantial positional perturbation. Edge residuals consistently decrease after optimization.

6.7 Hyperparameter Sensitivity

Four ablation studies characterize the optimizer’s behaviour.

TABLE 6.8: Effect of GPS prior weight w_{gps} (data1, ORB, 3 m bias, $w_{\text{match}} = 0.25$).

w_{gps}	Avg corr (m)	Eval err (m)	Res. before	Res. after	Seam SSIM
1	0.410	1.951	4.26	3.01	0.958
3	0.187	1.744	4.26	3.47	0.957
9	0.073	1.644	4.26	3.87	0.957
27	0.026	1.607	4.26	4.10	0.957

6.7.1 GPS Prior Weight

Reducing w_{gps} from 27 to 1 increases mean correction from 0.026 m to 0.410 m but raises evaluated pose error to 1.951 m (vs. 1.644 m at default), indicating over-correction. The default $w_{\text{gps}} = 9$ offers a practical compromise.

6.7.2 Match Weight

TABLE 6.9: Effect of match weight w_{match} (data1, ORB, 3 m bias, $w_{\text{gps}} = 9.0$).

w_{match}	Avg corr (m)	Eval err (m)	Res. before	Res. after	Seam SSIM
0.05	0.016	1.599	4.26	4.16	0.957
0.10	0.031	1.611	4.26	4.07	0.957
0.25	0.073	1.644	4.26	3.87	0.957
0.50	0.134	1.696	4.26	3.63	0.957
1.00	0.234	1.786	4.26	3.35	0.957

6.7.3 Feature Count

ORB is $7\times$ – $9\times$ faster than SIFT across all feature budgets with comparable edge counts and indistinguishable SSIM. The default of 700 features is a near-saturation point.

TABLE 6.10: Effect of feature count on optimizer runtime and quality (data1, clean trajectory).

Features	Opt. time (ms)		Edges		Seam SSIM	
	ORB	SIFT	ORB	SIFT	ORB	SIFT
200	17	152	1 014	1 174	0.9565	0.9567
500	23	212	1 265	1 381	0.9566	0.9569
700	27	256	1 333	1 432	0.9568	0.9567
1000	35	322	1 373	1 478	0.9569	0.9567

6.7.4 Solve Interval

TABLE 6.11: Effect of solve interval on runtime and correction (data1, clean, ORB).

Solve interval	Avg opt (ms)	Avg total (ms)	Edges	Seam SSIM
1	32	134	1 333	0.9579
3	28	129	1 333	0.9578
5	27	127	1 333	0.9568
10	26	125	1 333	0.9572

6.8 Comparison with Existing Pipelines

Table 6.12 compares the proposed system against OpenDroneMap (offline SfM) and Map2DFusion (incremental SLAM-based).

On data1, the proposed system achieves an $8.7\times$ wall-clock speedup over ODM. On data3, the gap widens to $87\times$. ODM produces finer GSD (4–10 cm/px vs. 50 cm/px) with global bundle adjustment. Map2DFusion is faster on data2 (48 ms vs. 137 ms per frame) but requires GPU hardware and a SLAM front-end. The proposed system occupies the practical middle ground: CPU-only, no SLAM, real-time throughput.

6.9 Discussion

Runtime versus accuracy trade-off. The experiments reveal a predictable cost–accuracy spectrum. Raw GPS placement achieves 6.8–18.8 FPS. Adding ORB optimization reduces throughput to 4.3–7.3 FPS but provides positional correction. SIFT

TABLE 6.12: Comparison with existing pipelines. Our system reports the Opt. ORB variant.

Dataset	Pipeline	Total time	Per-frame (ms)	GSD (cm/px)
data1	ODM (offline)	6m 33s	1 631	10.3
	Ours (real-time)	45s	188	50.0
data2	Map2DFusion	19.5s	48	31.8
	Ours (real-time)	56s	137	50.0
data3	ODM (offline)	46m 14s	19 004	4.0
	Ours (real-time)	32s	232	50.0

further reduces throughput to 1.8–2.3 FPS with marginal quality improvement. For CPU-constrained deployments, ORB is the practical default.

Optimizer conservatism. The translation-only optimizer is deliberately conservative. With the default $w_{\text{gps}} = 9.0$ and $w_{\text{match}} = 0.25$, the GPS prior dominates. The w_{gps} ablation shows that reducing it to 1.0 increases corrections $5.6\times$ but also increases evaluated pose error by 19%. The chosen defaults ensure the optimizer cannot degrade an already good trajectory.

Limitations. The pipeline fixes gimbal rotations and corrects only horizontal translations, so attitude errors propagate uncorrected. The flat-earth assumption introduces parallax artefacts near tall structures. The seam SSIM metric captures tile-boundary continuity but not sub-pixel geometric accuracy. The system does not produce georeferenced output in standard GIS formats.

Relation to offline pipelines. ODM produces orthomosaics at 4–10 cm/px GSD with global bundle adjustment but requires 6–46 minutes. Map2DFusion achieves the fastest per-frame time but requires GPU and SLAM. The two paradigms are complementary: the real-time mosaic guides the mission, and offline reconstruction follows for archival quality.

6.10 Conclusion

This thesis presented a CPU-only UAV orthomosaicking system that incrementally constructs georeferenced maps during flight. The three core contributions are:

1. A **translation-only pose graph** that corrects GPS drift by solving sparse pairwise constraints derived from ground-projected feature correspondences, achieving 4–8% residual reduction with only 27–80 ms overhead using ORB features.

2. A **view-aware compositor** that detects heading conflicts and arbitrates overlap using nadir preference, suppressing double-imaging artefacts without a DSM.
3. A **reproducible evaluation harness** demonstrating 4.3–18.8 FPS on three diverse datasets with seam SSIM above 0.91, stability under synthetic drift up to 4 m, and up to 87× speedup over offline SfM.

Beyond the algorithmic contributions, the thesis described two complete application systems: the Drone Mapper web application and the SkyForge Ground Control Station, the latter deployed for field trials with the Indian Army. The MAVLink deployment pipeline enables real-time mapping from any MAVLink-capable airframe through DJI-compatible XMP metadata injection.

6.11 Future Work

Several directions for future development are identified:

Adaptive GPS weighting. Dynamically adjusting w_{gps} based on GPS quality indicators (HDOP, satellite count) would permit larger corrections when visual evidence is strong and GPS is unreliable, while maintaining conservative behaviour under good GPS conditions.

DEM-aware projection. Replacing the flat-earth assumption with a Digital Elevation Model lookup (SRTM at 30 m or ALOS at 12.5 m resolution) for the ground intersection would reduce relief displacement artefacts:

$$\lambda = -\frac{C_z - \text{DEM}(x_{\text{est}}, y_{\text{est}})}{r_{w,z}}, \quad (6.1)$$

with 2–3 iterative refinement steps to resolve the coupling between the intersection point and lookup location.

GPU-accelerated blending. Porting the CPU-bound NumPy/OpenCV pipeline to CuPy or NVIDIA VPI would enable 4K processing at 30+ FPS on edge GPUs (e.g., NVIDIA Jetson Orin), essential for high-resolution field deployment.

Deep semantic segmentation. A lightweight segmentation network (MobileNetV3-UNet or SegFormer-B0 [21]) could classify map regions into navigable water, dry ground, vegetation, and structures, maintained as a parallel probability layer within the tiled map.

Loop closure and map deformation. Extending the pose graph with bag-of-visual-words loop closure detection and non-rigid tile deformation would correct accumulated drift on long missions without pixel-level re-stitching.

Standard GIS output. Adding GeoTIFF export with proper CRS metadata would enable integration with standard GIS tools (QGIS, ArcGIS) for operational use.

Bibliography

- [1] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [2] OpenDroneMap Community, “OpenDroneMap: The missing guide,” <https://opendronemap.org/>, 2023.
- [3] Pix4D, “Pix4D mapper technical documentation,” <https://www.pix4d.com/>, 2022.
- [4] Y. Bu *et al.*, “Map2DFusion: Real-time incremental UAV image mosaicing based on monocular SLAM,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 4564–4571.
- [5] A. Kern, M. Bobbe, Y. Khedar, and R. Bestmann, “OpenREALM: Real-time mapping for unmanned aerial vehicles,” in *International Conference on Unmanned Aircraft Systems (ICUAS)*, 2020.
- [6] N. Snavely, S. M. Seitz, and R. Szeliski, “Photo tourism: Exploring photo collections in 3D,” in *ACM SIGGRAPH*, 2006, pp. 835–846.
- [7] Agisoft, “Agisoft Metashape user manual,” <https://www.agisoft.com/>, 2023.
- [8] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multi-map SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [9] T. Qin, P. Li, and S. Shen, “VINS-Mono: A robust and versatile monocular visual-inertial state estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [10] E. Honkavaara *et al.*, “Georeferencing and orthoimage processing of airborne photogrammetric imagery,” *Photogrammetric Journal of Finland*, 2006.
- [11] M. Cramer, D. Stallmann, and N. Haala, “Direct georeferencing using GPS/Inertial exterior orientations for photogrammetric applications,” *International Archives of Photogrammetry and Remote Sensing*, vol. 33, pp. 198–205, 2000.
- [12] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, “g2o: A general framework for graph optimization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011.
- [13] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, “A tutorial on graph-based SLAM,” *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2010.

- [14] P. Burt and E. Adelson, “The Laplacian pyramid as a compact image code,” *IEEE Transactions on Communications*, vol. 31, no. 4, pp. 532–540, 1983.
- [15] M. Brown and D. G. Lowe, “Automatic panoramic image stitching using invariant features,” *International Journal of Computer Vision*, vol. 74, no. 1, pp. 59–73, 2007.
- [16] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [17] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [18] C. C. Paige and M. A. Saunders, “LSQR: An algorithm for sparse linear equations and sparse least squares,” *ACM Transactions on Mathematical Software*, vol. 8, no. 1, pp. 43–71, 1982.
- [19] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 2564–2571.
- [20] MAVLink Development Team, “MAVLink micro air vehicle communication protocol,” <https://mavlink.io/en/>, 2024.
- [21] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, “SegFormer: Simple and efficient design for semantic segmentation with transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.